

Advanced Java Programming

Module 4

Overview of Design Patterns & The MVC Design Pattern

Design Patterns

- A pattern is a proven solution to a problem in a context.
- Christopher Alexander says each pattern is a three-part rule which expresses a relation between a certain **context, a problem, and a solution**.
- Design patterns represent a solutions to problems that arise when developing software within a particular context.
- Design patterns represent the best practices used by experienced **object-oriented software** developers.
- Design patterns are solutions to general problems that software developers faced during software development.
- These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.
- i.e Patterns = problems & solution pairs in a context

Background

- Started in 1987 by Ward Cunningham and Ken Beck who were working with Smalltalk and designing GUIs.
- Popularized by Gamma, Helm, Johnson and Vlissides (The gang of four, Go4)
- The three of Go4 were working on frameworks (E++, Unidraw, HotDraw)
- Design pattern use a consistent documentation approach
- Design pattern are granular and applied at different levels such as frameworks, subsystems and sub-subsystems
- Design patterns are often organized as **creational, structural or behavioral**

Categorizing Pattern

Patterns, then, represent expert solutions to recurring problems in a context and thus have been captured at many levels of abstraction and in numerous domains. Numerous categories are:

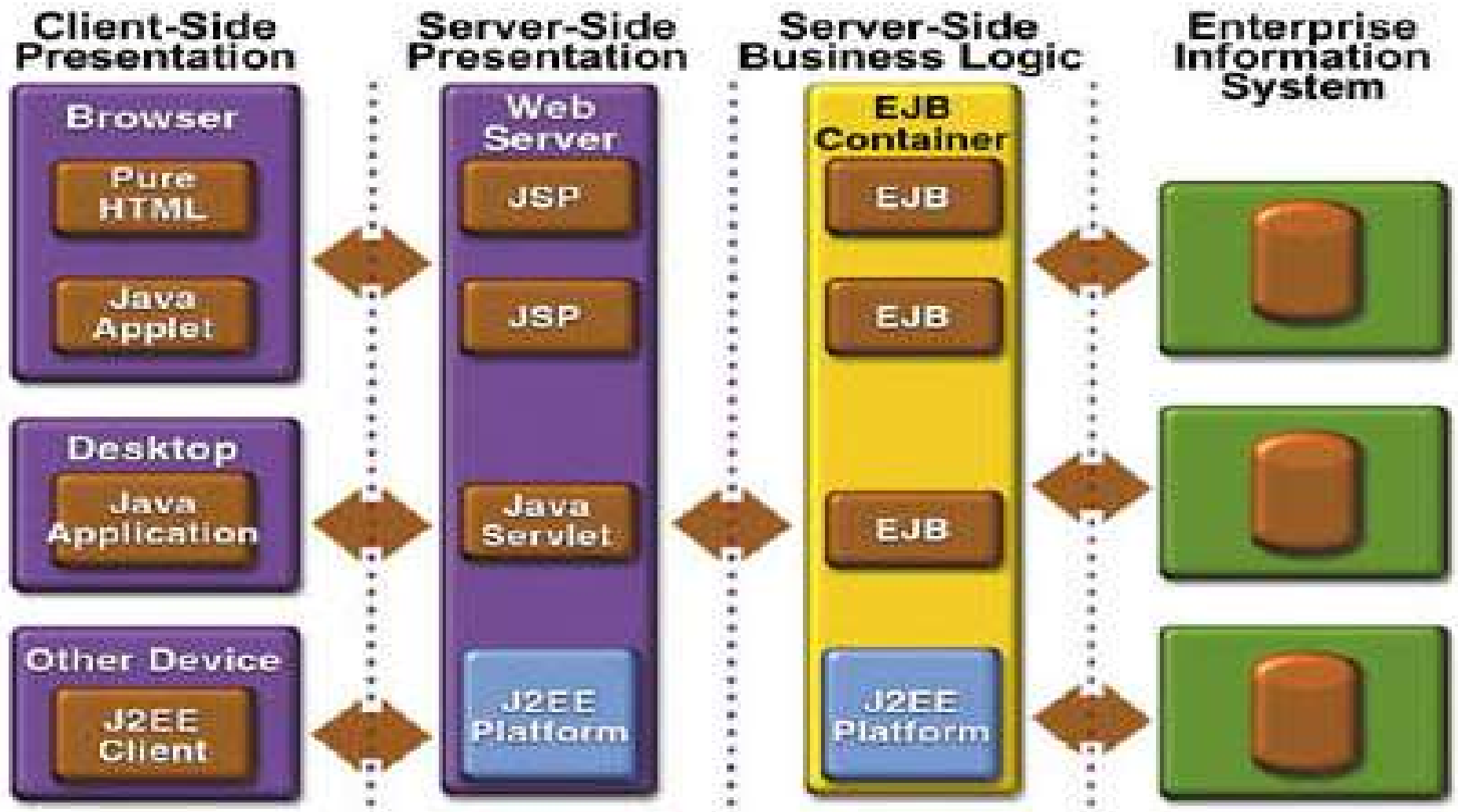
- Design
- Architectural
- Analysis
- Creational
- Structural
- Behavioral

Sun's J2EE Framework

- Components Containers and Connectors: **Hiding Complexity, Enhancing Portability**
- **Components** are the key focus of application developers
- **Containers** intercede between clients and components, providing services transparently to both, including **transaction support and resource pooling**.
- Connectors sit beneath the J2EE platform, defining a **portable service API** to plug into existing enterprise vendor offerings.

J2EE

- Components
 - Enterprise Java Beans (EJB)
 - Java Server Pages (JSP)
 - Servlets
- Containers (service providers)
 - Web container
 - Bean Container
- Connectors (connection service providers)



J2EE and Design Patterns

- J2EE: AN OPERATING SYSTEM FOR THE WEB Enterprise web applications, which live on networks and are accessible through browsers, are **redefining Enterprise Web Software**. This is the next wave of computing.
- The J2EE architecture is built to enable component developers to use a **Model View Controller (MVC) Design Pattern**.

Details of MVC Design Pattern

- **Name** (essence of the pattern)
- **Model View Controller MVC**
- **Context** (where does this problem occur)
- **MVC** is an architectural pattern that is used when developing interactive application such as a shopping cart on the Internet.
- **Problem** (definition of the reoccurring difficulty)
- User interfaces change often, especially on the internet where **look-and-feel** is a competitive issue. Also, the same information is presented in different ways. **The core business logic and data is stable.**

What is MVC?

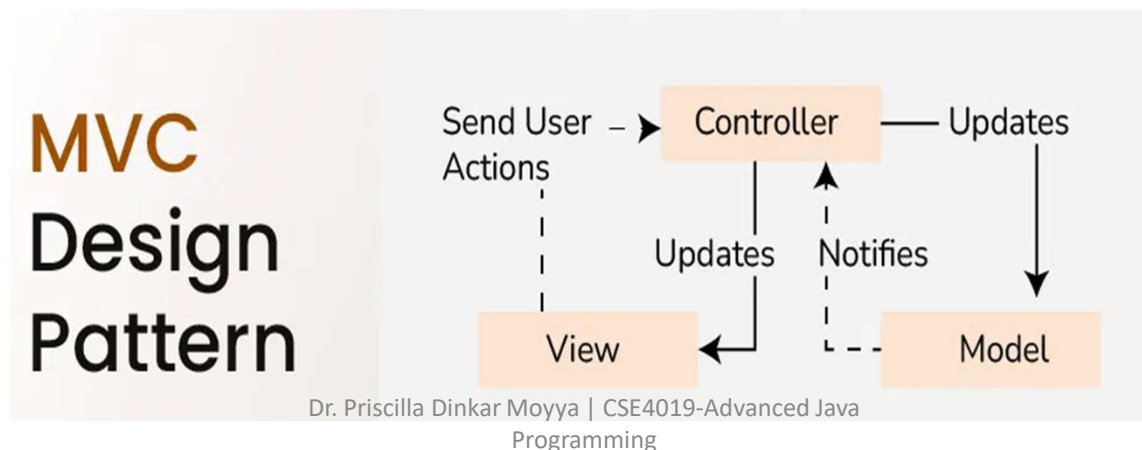
- MVC is an acronym for Model View Controller
- It represents a software design pattern developed at Xerox PARC in **1978 (!)**
- It explains a method of separating the visual, interaction and data components.
- Very popular, used extensively in Java and other languages.

Why use MVC?

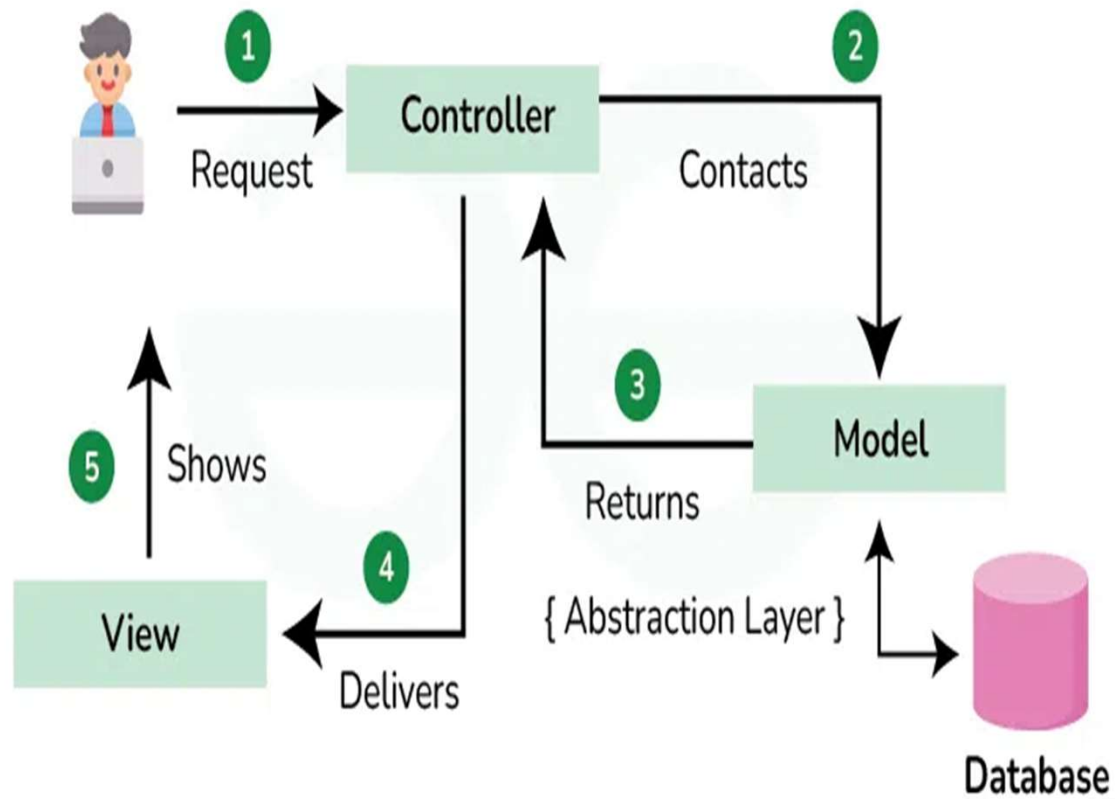
- Makes it very easy to have multiple different displays of the same information.
- For example: a graph and a table could both display and edit the same data.
- Essentially provides greater control over the UI and its behaviour.

MVC Design Pattern

- The MVC design pattern is a software architecture pattern that separates an application into three main components:
 - Model, View, and Controller, making it easier to manage and maintain the codebase.
 - It also allows for the reusability of components and promotes a more modular approach to software development modular approach to software development



MVC Design Pattern



MVC Design Pattern

- **User Interaction with View:** The user interacts with the View, such as clicking a button or entering text into a form.
- **View Receives User Input:** The View receives the user input and forwards it to the Controller.
- **Controller Processes User Input:** The Controller receives the user input from the View. It interprets the input, performs any necessary operations (such as updating the Model), and decides how to respond.
- **Controller Updates Model:** The Controller updates the Model based on the user input or application logic.
- **Model Notifies View of Changes:** If the Model changes, it notifies the View.

MVC – The Model

- The “Model” contains the data
- Has methods to access and possibly update its contents.
- Often, it implements an interface which defines the allowed model interactions.
- Implementing an interface enables models to be pulled out and replaced without programming changes.

MVC – The Controller

- Users interact with the controller.
- It interprets mouse movement, clicks, keystrokes, etc
- Communicates those activities to the model – eg: delete row, insert row, etc
- It's interaction with the model indirectly causes the View(s) to update

MVC – The View

- The View provides a visual representation of the model.
- There can be multiple views displaying the model at any one time.
- For example, a companies finances over time could be represented as a table and a graph.
- These are just two different views of the same data.
- When the model is updated, all Views are informed and given a chance to update themselves.

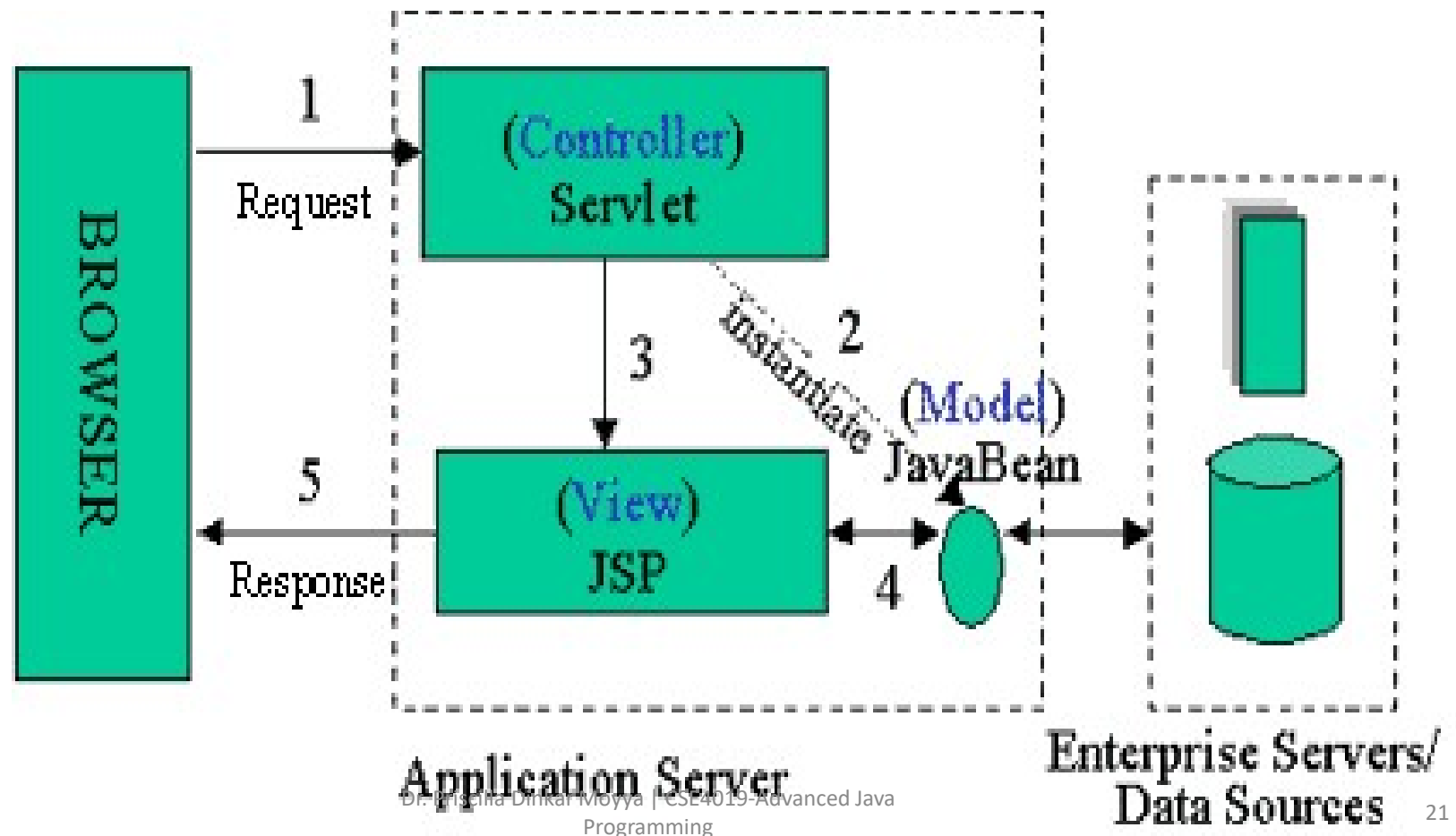
When to Use the MVC Design Pattern

- **Complex Applications:** Use MVC for apps with many features and user interactions, like e-commerce sites. It helps organize code and manage complexity.
- **Frequent UI Changes:** If the UI needs regular updates, MVC allows changes to the View without affecting the underlying logic.
- **Reusability of Components:** If you want to reuse parts of your app in other projects, MVC's modular structure makes this easier.
- **Testing Requirements:** MVC supports thorough testing, allowing you to test each component separately for better quality control.

MVC continued

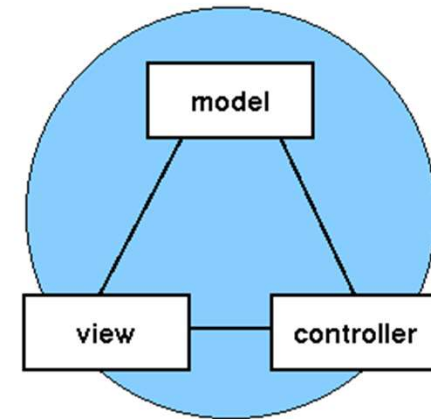
- **Solution** (how do you solve the problem)
- Use the software engineering principle of “separation of concerns” to divide the application into three areas:
 - **Model** encapsulates the **core data and functionality**
 - **View** encapsulates the presentation of the data there can be many **views of the common data**
 - **Controller** accepts input from the user and makes request from the model for the data to produce a new view.

MVC Structure for J2EE



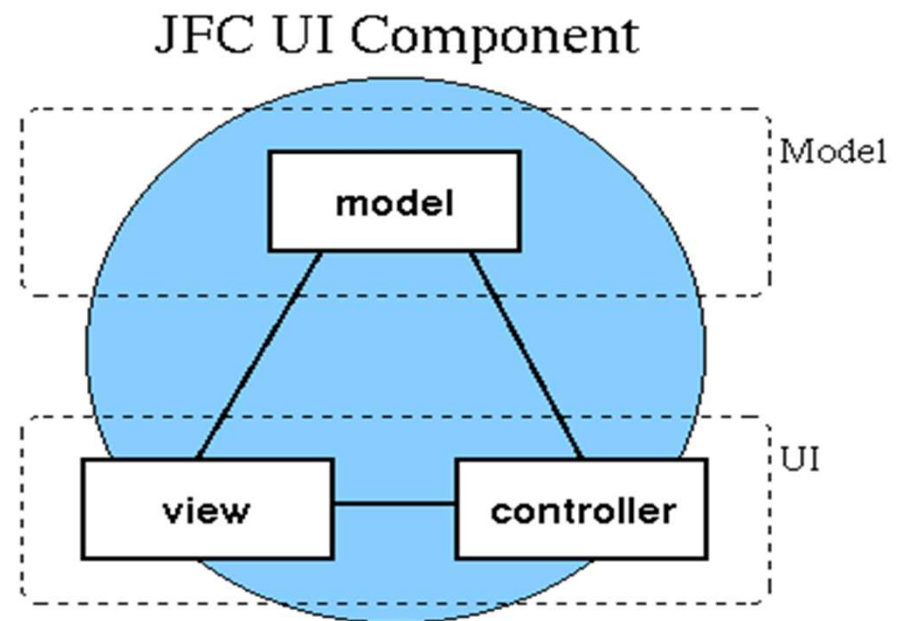
MVC Architecture

- The **Model** represents the structure of the data in the application, as well as application-specific operations on those data.
- A **View** (of which there may be many) presents data in some **form** to a user, in the context of some application function.
- A **Controller** translates user actions (mouse motions, keystrokes, words spoken, etc.) and user input into application function calls on the model, and selects the appropriate View based on user preferences and Model state.



Example of MVC Design Pattern

- Let's investigate this statement by looking at a small application that demonstrates MVC on J2EE



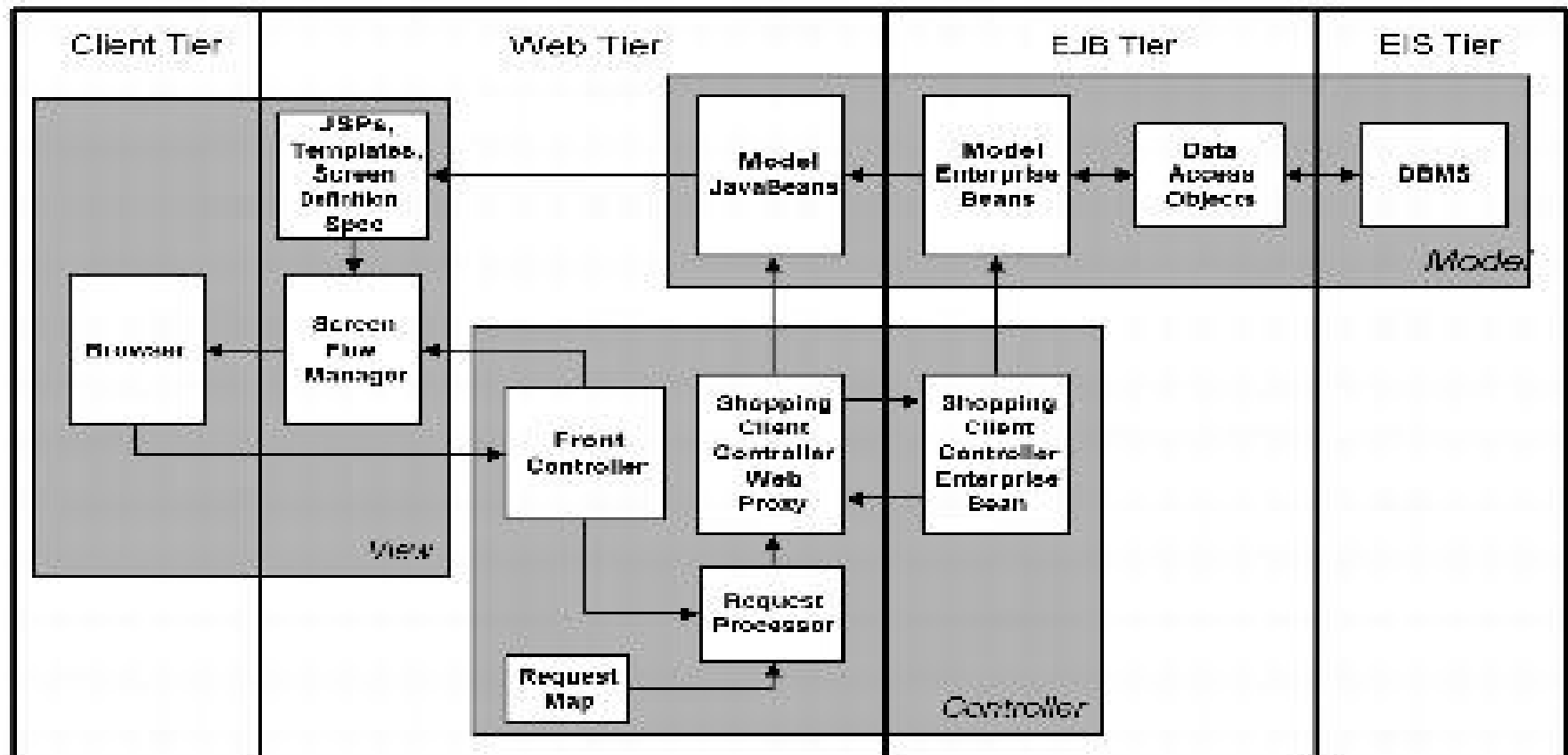
Java Pet Store- MVC Design Pattern

- The Java Pet Store is a reference application that demonstrates J2EE technologies.
- It demonstrates interaction between Java Server Pages (JSP's), custom Tag Libraries, JavaBeans, and Enterprise Java Beans.
- It demonstrates a real-world approach to application development, where the presentation of data is separated from the process of obtaining data from objects which interact with the enterprise or database tier.
- The Pet Store application implements MVC (Model-View-Controller) design, and demonstrates one way to design an application that should scale well.

Multi Tier Architecture

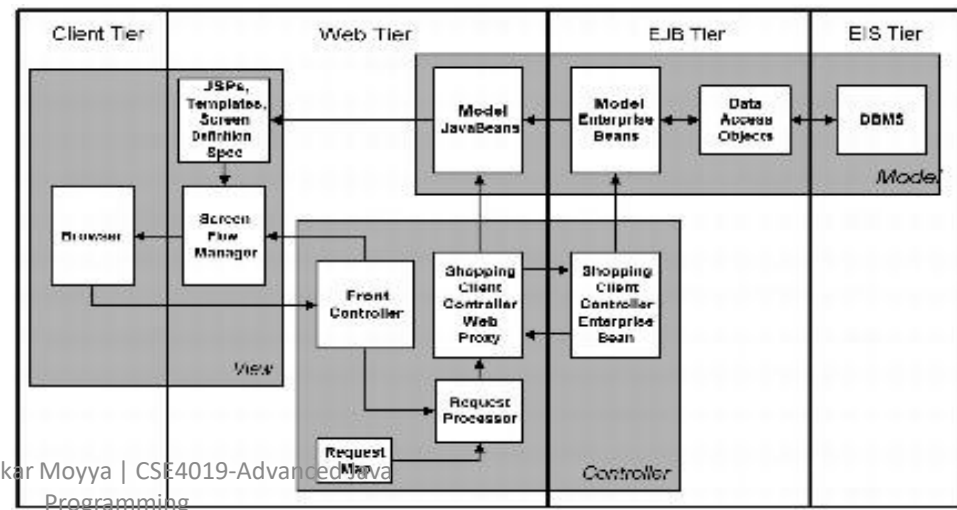
- The Java Pet Store design is divided into multiple tiers:
 - A. Client tier
 - B. Web tier
 - C. Enterprise JavaBeans tier
 - D. Enterprise Information System tier.
- These tiers are not necessarily arranged hierarchically.
- Each tier may communicate directly with other tiers, or indirectly by way of intermediate tiers.

J2EE Architecture Tiers



A. Details of Client Tier

- The Client tier is responsible for presenting data to the user, interacting with the user, and communicating with the other tiers of the application.
- The Client tier is the only part the application the user ever sees.
- The Client tier communicates with other tiers by way of well-defined interfaces.
- A separate Client tier in the design provides flexibility and extensibility.

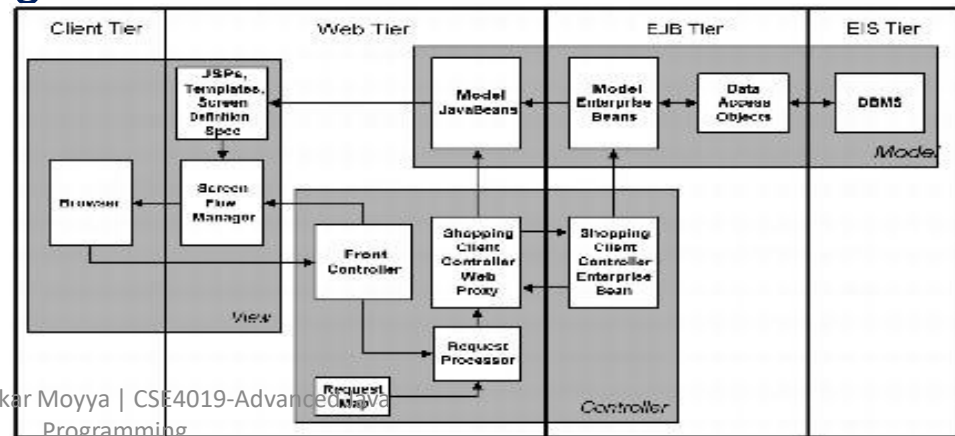


A. Details of Client Tier

- In The Java Pet Store Client tier consists mainly of a browser displaying Web pages generated from server-side JSP pages in the Web tier.
- Future new clients can be written using technologies or languages that do not yet even exist, since they must conform only to the interface for communicating with other tiers

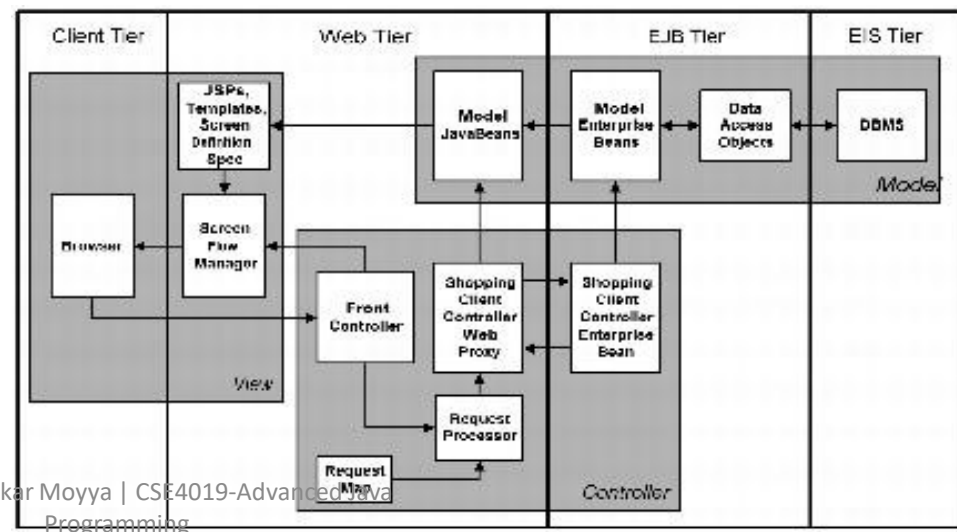
B. Web Tier

- The Web tier is responsible for performing all Web-related processing, such as serving HTML, instantiating Web page templates, and formatting JSP pages for display by browsers.
- The Web tier in the Java Pet Store does all of these, and takes on the Controller functions for the Web application, caching model data interpreting user inputs, selecting appropriate Views based on application flow, and managing database connections.



C. EJB Tier

- Enterprise JavaBeans are software business components which extend servers to perform application-specific functionality.
- The interface between these components and their containers is defined in the EJBs specification.
- Essentially, the EJBs tier provides a component model for access to distributed system services and persistent data.



C. EJB Tier

- Both stand-alone clients and Web applications in the Web tier can use EJB components hosted by the EJBs tier.
- It also simplifies application component development, because details about system issues such as persistence, reentrancy, transactions, remote access, and so on, are all handled by the container.

D.Enterprise Information System (EIS) Tier

- The EIS tier is the enterprise information infrastructure.
- Members of the EIS tier typically include enterprise information planning (ERP) systems, transaction processing monitors, relational database management systems, and legacy enterprise applications.
- Access to the EIS tier is usually transactional, to ensure that data are consistent across application boundaries.
- The EIS tier also enforces **security and offers scalability**.

MVC supports Modular Design

- Has set of modules, each tightly coupled internally, and loosely coupled between modules.
- Each module has an interface that defines the module's functional requirements and provides a place where third-party products may be integrated.
- The Java Pet Store demo modules are:
 - User Account
 - Product Catalog
 - Order Processing
 - Messaging
 - Inventory
 - Control
- The **Modular design** supports the design goal of **reusable software**.

Java Pet store- MVC

- **Views**
 - JSP pages, composed with templates and displayed in an HTML browser
- **Controller**
 - maps user input from the browser to request events, and forwards those events to the Shopping Client Controller in the EJB tier.
- **Model**
 - EJB Tier

MVC Details in Java Pet store

Model represents the structure of the data in the application, as well as application-specific operations on data

- - CartModel, InventoryModel, CustomerEJB, and others

Views are Java server pages (JSPs)

- rendered from the web container to the browser, stand-alone applications that provide View functionality, and interfaces to spreadsheet programs, such as the StarOffice™ suite.

Controller is server side java program (Servlet)

- MainServlet.java, which dispatches browser requests to other controller objects, such as ShoppingClientController.java, AdminClientController.java, and their related support classes.

Views:Java Server Page (JSP)

- <http://java.sun.com/products/jsp/>
- Technology for developing dynamic web sites that replaces CGI
- Thought of as a server-side scripting tool
- Contains HTML and Java code (scripts)
- Is compiled into a servlet and executes on the server.

JSP Example



Running on Java 2 Enterprise Edition Reference Implementation

ShoppingCart.jsp

- Java Server Pages (JSP)



The screenshot shows the 'Java Pet Store demo' interface. At the top, there's a header with a parrot logo, the store name, a search bar, and links for 'Sign-in' and 'What are you looking for?'. Below the header, there are category links: 'Fish | Dogs | Reptiles | Cats | Birds'. The main section is titled 'Shopping Cart:'. It contains a table with columns: Item ID, Product Name, In Stock, Unit Price, Quantity, and Total Cost. There are two items in the cart: 'Tiger Shark' (EST-3) and 'Iguana' (EST-13). Each item has a 'Remove' button. The total cost is \$111.00. There are 'Update Cart' and 'Proceed to checkout' buttons.

	Item ID	Product Name	In Stock	Unit Price	Quantity	Total Cost
Remove	EST-3	Tiger Shark	yes	\$18.50	6	\$92.50
Remove	EST-13	Iguana	yes	\$18.50	1	\$18.50
Total:						\$111.00

[Update Cart](#)

[Proceed to checkout](#)

Controller Servlet

- A java class that runs on the server
- Extends http Servlet
- Runs in a container class
(servlet/JSP engine)
- Application servers (Jrun, WebLogic) have the containers
- This has the logic for the application

EJBs

- Enterprise Java Beans Connect Servlets to the back end database
- Examples of EJBs in Java Pet store are:
AccountHandler, ModelUpdateManager, ShoppingClientControllerHome,
CartHandler,
ShoppingClientControllerEJB, SigninHandler, OrderHandler

Advantages of MVC

- Separating Model from View (that is, separating data representation from presentation)
 - easy to add multiple data presentations for the same data,
 - facilitates adding new types of data presentation as technology develops.
 - Model and View components can vary independently enhancing maintainability, extensibility, and testability.

Advantages of MVC design Pattern

- Separating Controller from View (application behavior from presentation)
 - permits run-time selection of appropriate Views based on workflow, user preferences, or Model state.
- Separating Controller from Model (application behavior from data representation)
 - allows configurable mapping of user actions on the Controller to application functions on the Model.

Consequences or Benefits

- We make changes without bringing down the server.
- We leave the core code alone
- We can have multiple versions of the same data displayed
- We can test our changes in the actual environment.
- We have achieved “separation of concerns”

Implementation of MVC using Java

- **Implementation of MVC using Java** - To implement MVC pattern in Java, we are required to create the following three classes.
-
- **Employee Class**, will act as model layer
- **EmployeeView Class**, will act as a view layer
- **EmployeeController Class**, will act a controller layer

Implementation of MVC using Java

Employee.java

```
// class that represents model
public class Employee {
    // declaring the variables
    private String EmployeeName;
    private String EmployeeId;
    private String EmployeeDepartment;
    // defining getter and setter methods
    public String getId() { return EmployeeId; }
    public void setId(String id) { this.EmployeeId = id; }
    public String getName() { return EmployeeName; }
    public void setName(String name) { this.EmployeeName = name; }
    public String getDepartment() { return EmployeeDepartment; }
    public void setDepartment(String Department)
    { this.EmployeeDepartment = Department; }
}
```

The above code simply consists of getter and setter methods to the Employee class.

Implementation of MVC using Java

EmployeeView.java

// class which represents the view

public class EmployeeView

{

 // method to display the Employee details

View Layer - As the name depicts, view **represents the visualization of data** received from the model. The view layer **consists of output** of application or user interface. It **sends the requested data** to the client, that is fetched from model layer by controller. Example where we create a view using the EmployeeView class.

public void printEmployeeDetails (String EmployeeName, String EmployeeId, String EmployeeDepartment)

{

 System.out.println("Employee Details: ");

 System.out.println("Name: " + EmployeeName);

 System.out.println("Employee ID: " + EmployeeId);

 System.out.println("Employee Department: " + EmployeeDepartment);

}

}

4/6/2026

Implementation of MVC using Java

EmployeeController.java

// class which represent the controller

public class EmployeeController

{

// declaring the variables model and view

private Employee model;

private EmployeeView view;

// constructor to initialize

public EmployeeController(Employee model, EmployeeView view)

{

this.model = model;

this.view = view;

}

Implementation of MVC using Java

```
// getter and setter methods
    public void setEmployeeName(String name) { model.setName(name); }
    public String getEmployeeName() { return model.getName(); }
    public void setEmployeeId(String id) { model.setId(id); }
    public String getEmployeeId() { return model.getId(); }
    public void setEmployeeDepartment(String Department)
    { model.setDepartment(Department); }
    public String getEmployeeDepartment() { return model.getDepartment(); }
// method to update view
    public void updateView()
{ view.printEmployeeDetails(model.getName(), model.getId(), model.getDepartment()); }
}
```


Implementation of MVC using Java

MVCMain.java

// main class

```
public class MVCMain {  
    public static void main(String[] args)  
    {  
        // fetching the employee record based on the employee_id from the database  
        Employee model = retrieveEmployeeFromDatabase();  
        // creating a view to write Employee details on console  
        EmployeeView view = new EmployeeView();  
        EmployeeController controller = new EmployeeController(model, view);  
        controller.updateView();  
        //updating the model data  
        controller.setEmployeeName("Bhopal");  
        System.out.println("\n Employee Details after updating: ");  
        controller.updateView();  
    }  
}
```

Implementation of MVC using Java

```
private static Employee retrieveEmployeeFromDatabase()
{
    Employee Employee = new Employee();
    Employee.setName("VIT");
    Employee.setId("112");
    Employee.setDepartment("SCSE");
    return Employee;
}
```

Output:

Employee Details:

Name: VIT

Employee ID: 111

Employee Department: SCSE

Employee Details after updating:

Name: Bhopal

Employee ID: 111

Employee Department: SCSE

The **MVCMain** class fetches the employee data from the method where we have entered the values. Then it pushes those values in the model. After that, it initializes the view (EmployeeView.java). When view is initialized, the Controller (EmployeeController.java) is invoked and bind it to Employee class and EmployeeView class. At last the updateView() method (method of controller) update the employee details to be printed to the console.

Swing

- Swing is Entirely written in Java.
- Java Swing Components are Platform-independent
- The Swing Components are lightweight.
- Swing Supports a Pluggable look and feel and Swing provides more powerful components.
- Swing provides a comprehensive set of components for building GUIs, including buttons, text fields, panels, and more.
- These components are highly customizable, allowing developers to create visually appealing and user-friendly interfaces.

MVC in Swing... Not quite

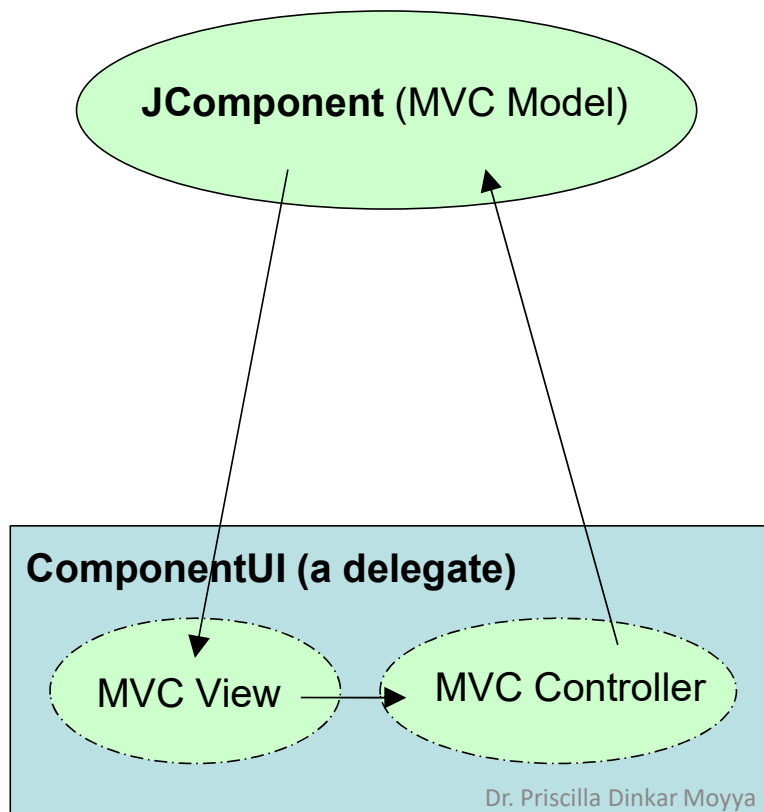
- Swing often merges the View and Controller together into one object.
- Sun call's this a delegate.
- The delegate interprets the user's interactions and updates the model
- It also handles the display
- Note that this still allows multiple views on the same model.
- It has nothing to do with C# delegates.

The MVC Power

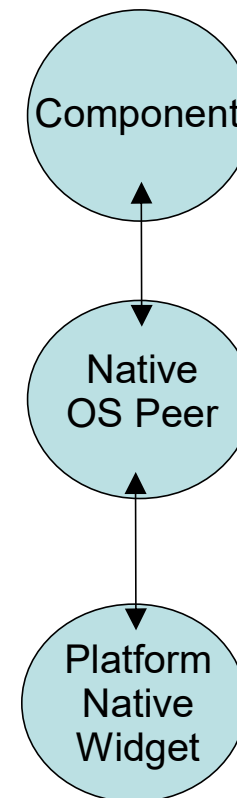
- One of the best examples of MVC/Delegates in Swing is the pluggable look and feel (aka PLAF)
- PLAF enables the entire UI look to be changed by simply switching PLAF's
- A simple matter of switching the delegate which draws buttons, lists, etc.
- Replaces AWT's heavyweight OS-look with a controllable, lightweight one.
- Tradeoff in performance. AWT was definitely faster.

PLAF MVC vs AWT

Swing's PLAF



AWT (not MVC!)



Beans Vs Swing

In Java, beans are reusable software components that can be visually manipulated, while Swing is a graphical user interface (GUI) toolkit:

- Beans**

- A Java class that is public and not abstract, with a public "null" constructor and properties defined by public getter and setter methods.
- Beans can be chosen from a palette of tools and manipulated graphically by an application designer.

- Swing**

- A GUI widget toolkit for Java that is an extension of AWT.
- Swing components are used to design different applications.
- Swing components display highly formatted information that can be modified by the user

MVC Analysis of Swing Buttons.

- The model-view-controller (MVC) architecture is used in the Swing framework to design buttons:
 - **Model** : Defines the button's behavior and internal state. The ButtonModel interface captures the model's behavior.
 - **View** : Creates the button's visual representation and handles user input. The ButtonUI interface captures the view and controller's behavior.
 - **Controller** : Translates user input into changes in the model. The controller mediates the flow of data between the model and view.

MVC Analysis of Swing Buttons.

- Here's how the MVC architecture works for a button:
 1. A user interacts with the view, such as by clicking a button.
 2. The view recognizes the action and calls the appropriate method on the controller.
 3. The controller accesses the model and may update it based on the user's action.
 4. If the model changes, it notifies the controller.
- The MVC architecture separates a program's logic into three independent parts, which can make applications easier to code, debug, and test.
- Swing components are considered superior to AWT components because they use the MVC architecture, which gives them greater functionality.

Layout Management

Layout Management

- The layout managers are used to arrange components in a particular manner
- Layout managers is an interface that is implemented by all the classes of layout managers.
- To become familiar with common user-interface components, such as radio buttons, check boxes, and menus
- A layout managers is an object that controls the size and the position of components in a container.
- Every container object has a LayoutManger object that controls its layout.

Layout Management

- The following are the frequently used list of layout manages:
- `Java.awt.BorderLayout`
- `Java.awt.FlowLayout`
- `Java.awt.GridLayout`
- `Java.awt.CardLayout`
- `Java.awt.GridBaglayout`

Border Layout

- The BorderLayout is used to arrange the components in five regions: NORTH, EAST, SOUTH, WEST, or CENTER.
- Each region (area) may contain one component only. It is the default layout of the frame or window.

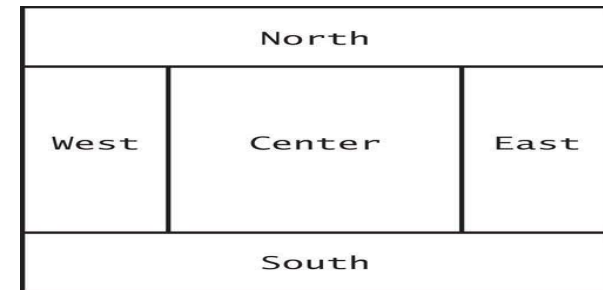
```
panel.setLayout(new BorderLayout());
```

```
panel.add(component, BorderLayout.NORTH);
```

- Examples of BorderLayout class:

Constructor:

- BorderLayout() – but no gaps between the components
- BorderLayout(int hgap, int vgap) – gaps between the components with hgap and vgap



Flow Layout

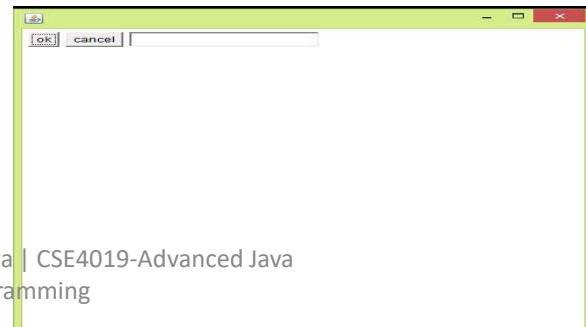
- The FlowLayout is used to arrange the components in a line, one after another (in a flow). It is the default layout of the applet or panel.
- LEFT, RIGHT, CENTER, LEADING, TRAILING

Constructor:

FlowLayout() – with centered alignment and a default 5 unit horizontal and vertical gap

FlowLayout(int align) – with given alignment and a default 5 unit horizontal and vertical gap

FlowLayout(int align, int hgap, int vgap) – with given alignment and given horizontal and vertical gap



Grid Layout

- Components are placed in boxes in a simple table arrangement. Specify the size (rows then columns) of the grid.
- Then add components which will be placed from the upper left, across, then down.

Constructor:

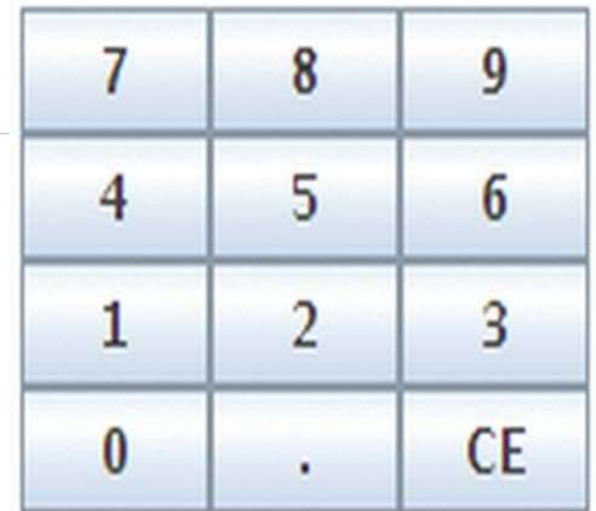
`GridLayout()` – one column per component in a row

`GridLayout(int rows, int columns)` – with rows x columns but no gap

`GridLayout(int rows, int columns, int hgap, int vgap)` – with rows x columns with hgap and vgap

Grid Layout

```
JPanel buttonPanel = new JPanel();  
buttonPanel.setLayout(new GridLayout(4, 3));  
buttonPanel.add(button7);  
buttonPanel.add(button8);  
buttonPanel.add(button9);  
buttonPanel.add(button4);  
...
```



7	8	9
4	5	6
1	2	3
0	.	CE

Figure 2 The Grid Layout

Card Layout

- The CardLayout is used to arrange the components and only one component is visible at a time.

Constructor:

CardLayout() – with zero horizontal and vertical gap

CardLayout(int hgap, int vgap) – with given horizontal and vertical gap



Card Layout

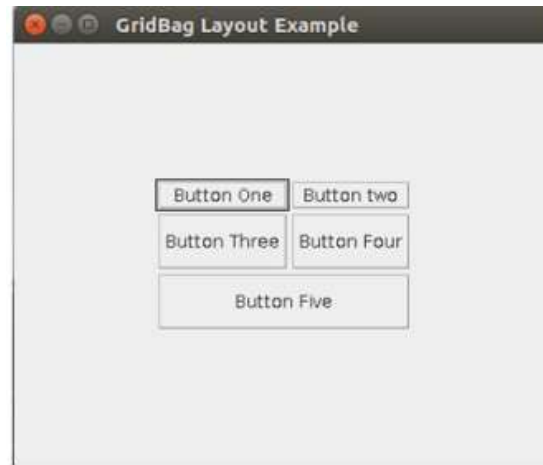
- Methods:
- `Public void next(Container parent)` – flip to next
- `Public void previous(Container parent)` – flip to previous
- `Public void first(Container parent)` – flip to first
- `Public void last(Container parent)` – flip to last
- `Public void show(Container parent, String name)` – flip to specific card with name

GridBag Layout

- The Java GridBagLayout class is used to align components vertically, horizontally or along their baseline.

Constructor

GridBagLayout(): The parameterless constructor is used to create a grid bag layout manager.



Components

Component Class

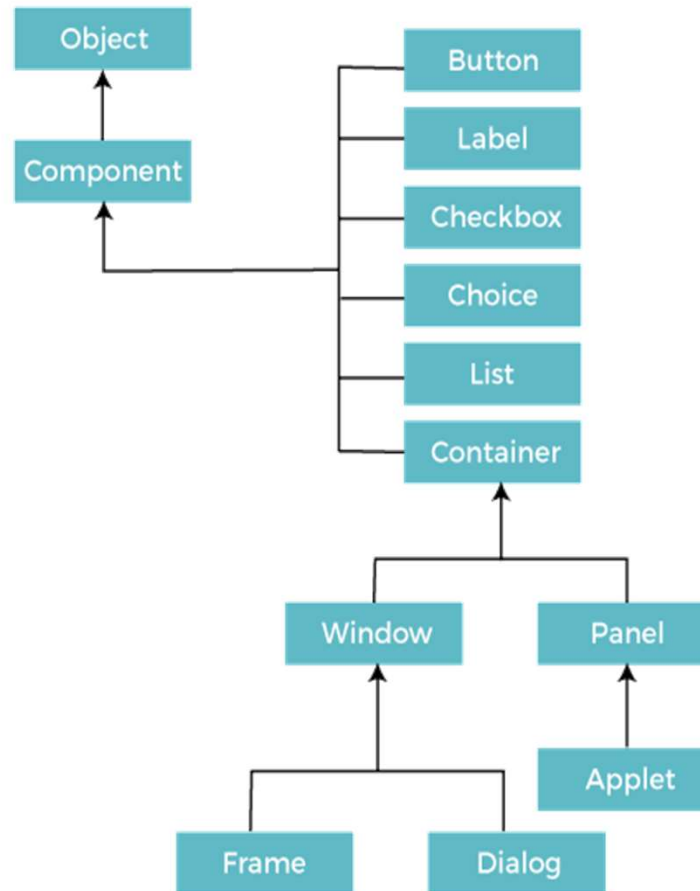
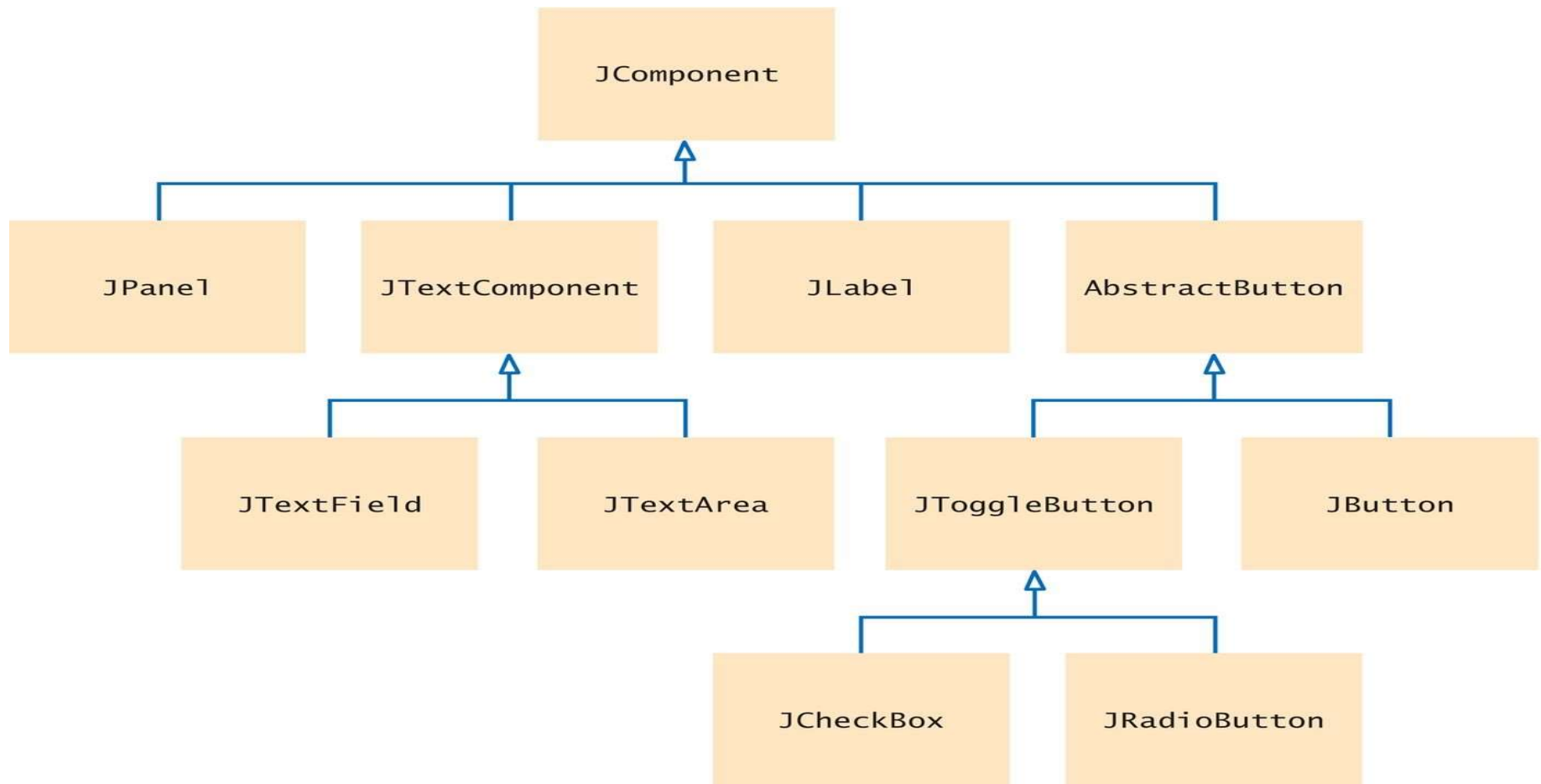


Figure 5 A Part of the Hierarchy of Swing User-Interface Components

Component Class



Container

- The Container is a component in AWT that can contain another components like buttons, textfields, labels etc. The classes that extends Container class are known as container such as Frame, Dialog and Panel.
- It is basically a screen where the components are placed at their specific locations. Thus it contains and controls the layout of components.
- There are four types of containers in Java AWT:
 - Window
 - Panel
 - Frame
 - Dialog

Container

Window

- The window is the container that have no borders and menu bars. You must use frame, dialog or another window for creating a window. We need to create an instance of Window class to create this container.

Panel

- The Panel is the container that doesn't contain title bar, border or menu bar. It is generic container for holding the components. It can have other components like button, text field etc. An instance of Panel class creates a container, in which we can add components.

Frame

- The Frame is the container that contain title bar and border and can have menu bars. It can have other components like button, text field, scrollbar etc. Frame is most widely used container while developing an AWT application.

Useful Methods of Component Class

Method	Description
<code>public void add(Component c)</code>	Inserts a component on this component.
<code>public void setSize(int width,int height)</code>	Sets the size (width and height) of the component.
<code>public void setLayout(LayoutManager m)</code>	Defines the layout manager for the component.
<code>public void setVisible(boolean status)</code>	Changes the visibility of the component, by default false.

Java Event classes and Listener interfaces

Event Classes	Listener Interfaces
ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

Events, Listeners, Adapters and Handler Methods

Event	Listener / Adapter	Handler Methods
ActionEvent	ActionListener	actionPerformed
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged
MouseEvent	MouseListener MouseAdapter	mouseClicked mouseEntered mouseExited mousePressed mouseReleased
KeyEvent	KeyListener KeyAdapter	keyPressed keyReleased keyTyped
ComponentEvent	ComponentListener ComponentAdapter	componentShown componentHidden componentMoved componentResized

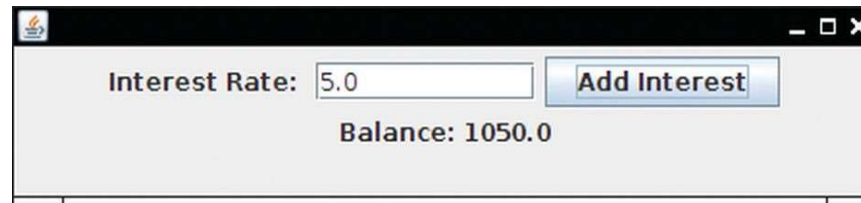
Registration Methods

- **Button**
 - `public void addActionListener(ActionListener a){}`
- **MenuItem**
 - `public void addActionListener(ActionListener a){}`
- **TextField**
 - `public void addActionListener(ActionListener a){}`
 - `public void addTextListener(TextListener a){}`
- **TextArea**
 - `public void addTextListener(TextListener a){}`
- **Checkbox**
 - `public void addItemListener(ItemListener a){}`
- **Choice**
 - `public void addItemListener(ItemListener a){}`
- **List**
 - `public void addActionListener(ActionListener a){}`
 - `public void addItemListener(ItemListener a){}`

Processing Text Input

- Dialog boxes allows for user input.
- Popping up a separate dialog box for each input is not a natural user interface.
- Most graphical programs collect text input through text fields. The JTextField class provides a text field.
- Specify the width for the text field.
- If the user exceeds this width, text will 'scroll' left.

```
final int FIELD_WIDTH = 10;  
final JTextField rateField = new JTextField(FIELD_WIDTH);
```



Dr. Priscilla Dinkar Moyya | CSE4019-Advanced Java
Programming

Figure 4 An Application with a Text Field

Add a Label and a Button

- Add a label to describe the field:

```
JLabel rateLabel = new JLabel("Interest Rate: ");
```

- Add a button for user to indicate input is complete.
- actionPerformed method can use getText to get input as a String.
- Convert to a numeric value if used for calculations.

```
class AddInterestListener implements ActionListener
{
    public void actionPerformed(ActionEvent event)
    {
        double rate = Double.parseDouble(rateField.getText()); double interest = balance * rate / 100;
        balance = balance + interest; resultLabel.setText("Balance: " + balance);
    }
}
```

Text Areas

Create multi-line text areas with a JTextArea object. Set the size in rows and columns.

```
final int ROWS = 10; // Lines of text
final int COLUMNS = 30; // Characters in each row
JTextArea textArea = new JTextArea(ROWS, COLUMNS);
```

- Use the setText method to set the text of a text field or text area.

```
textArea.append(balance + "\n");
```

Can use the text area for display purposes only.

```
textArea.setEditable(false);
```

Scrolling

- To add scroll bars, use JScrollPane:

```
JScrollPane scrollPane = new JScrollPane(textArea);
```

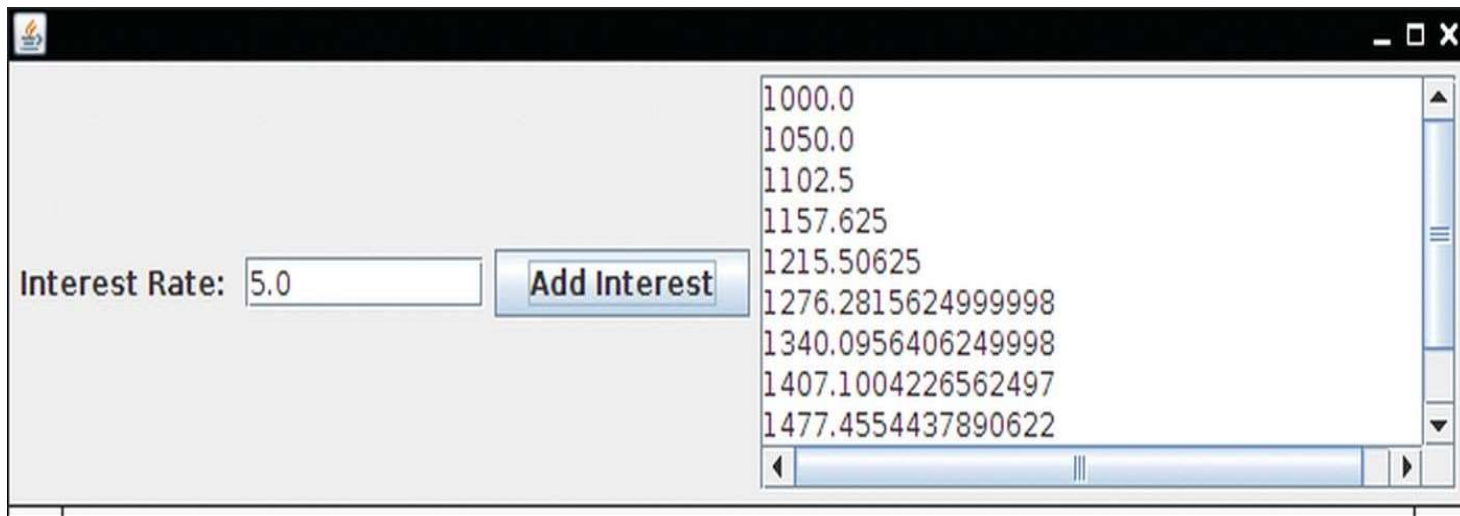


Figure 6 The Investment Application with a Text Area Inside Scroll Bars

Choices

GUI components for selections:

Radio Buttons For a small set of mutually exclusive choices. Check Boxes For a binary choice.

Combo Boxes For a large set of choices.

Radio Buttons



© Michele Cornelius/iStockphoto.

- Only one button in a set can be selected.
- Selecting a button clears previous selection.
- In an old fashioned radio, pushing down one station button released the others.
- Create each button individually.
- Add all buttons in the set to a ButtonGroup object:

```
JRadioButton smallButton = new JRadioButton("Small");  
JRadioButton mediumButton = new JRadioButton("Medium");  
JRadioButton largeButton = new JRadioButton("Large");  
ButtonGroup group = new ButtonGroup(); group.add(smallButton);  
group.add(mediumButton); group.add(largeButton);
```

Use `isSelected` to find out whether a button is selected, like:

```
if (largeButton.isSelected()) { size = LARGE_SIZE; }
```

Radio Button Panels

- Use a panel for each set of radio buttons.
- The default border for a panel is invisible (no border). You can add a border to a panel to make it visible.
- Also add a title.

```
JPanel panel = new JPanel();  
panel.add(smallButton);  
panel.add(mediumButton);  
panel.add(largeButton);  
panel.setBorder(new TitledBorder(new EtchedBorder(),"Size"));
```

User Interface Components

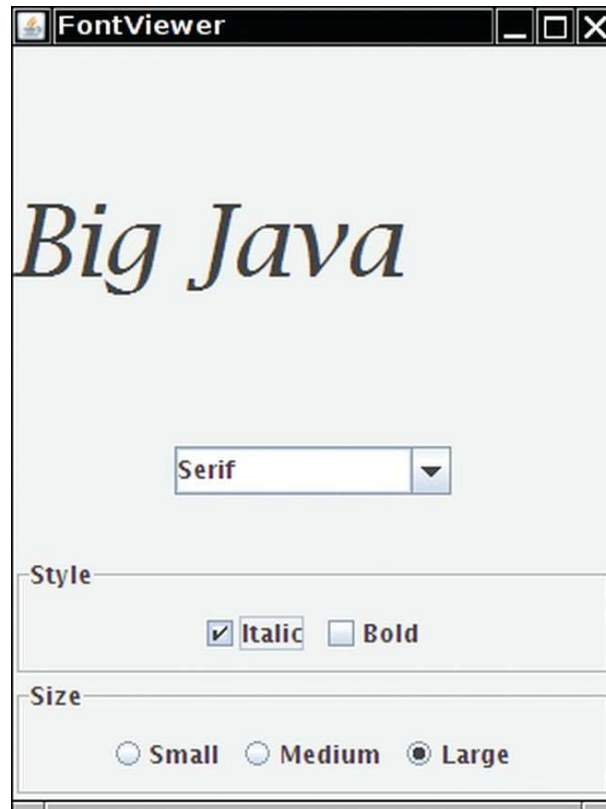


Figure 7 A Combo Box, Check Boxes, and Radio Buttons

Check Boxes

- A check box has two states: checked and unchecked. Use for choices that are not mutually exclusive.
 - For example in Figure 7, text may be Italic, Bold, both or neither.
- Because check box settings do not exclude each other, you do not need to place a set of check boxes inside a button group.
- Radio buttons are round and have a black dot when selected. Check boxes are square and have a check mark when selected. Construct a text box:

```
JCheckBox italicCheckBox = new JCheckBox("Italic");
```

Use `isSelected` to find out whether a check box is selected.

Combo Boxes

- A combo box is a combination of a list and a text field. Clicking the arrow to the right of the text field opens the list of selections.
- Use a combo box for a large set of choices.
- Use when radio buttons would take up too much space. It can be either:
 - Closed (shows one selection). Open, showing multiple selections.
- It can also be editable:
 - Type a selection into a blank line.

```
facenameCombo.setEditable();
```

Adding and Selecting Items

Add text 'items' to a combo box that will show in the list:

```
JComboBox facenameCombo = new JComboBox();  
facenameCombo.addItem("Serif");  
facenameCombo.addItem("SansSerif");  
...
```

- Use the `getSelectedItem` method to return the selected item (as an `Object`).
- Combo boxes can store other objects in addition to strings, so casting to a string may be required:



```
String selectedString = (String)  
facenameCombo.getSelectedItem();
```

4/6/2024 **Figure 8 An Open Combo Box** Dr. Priscilla Dinkar Moyya | CSE4019-Advanced Java Programming

Menu Bars and Menus

- A menu bar displays a list of top-level menu choices. Each choice is associated with a dropdown menu.
- This concept is implemented in Java by the following classes:
 - **MenuBar**, **Menu**, and **MenuItem**.
- In general, a menu bar contains one or more Menu objects. Each Menu object contains a list of MenuItem objects. Each MenuItem object represents something that can be selected by the user.

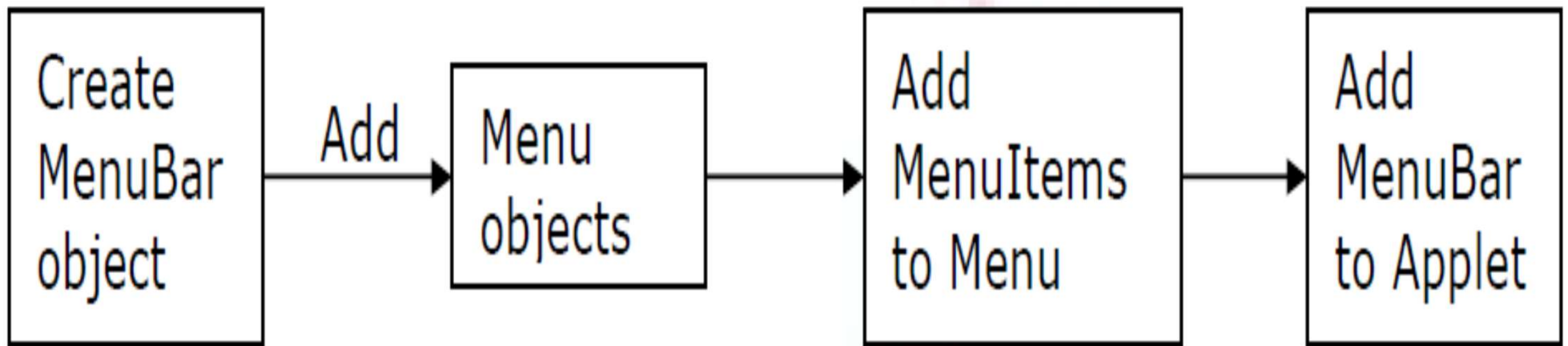


Fig. Creating a menu on Frame

- MenuBar Class Defines only default constructor.
- Menu Class Constructors
 - Menu()
 - Menu(String optionName)
 - Menu(String optionName, boolean removable)
- Here, *optionName* specifies the name of the menu selection.
- Individual menu items constructors:
 - MenuItem()
 - MenuItem(String *itemName*)
 - MenuItem(String *itemName*, MenuShortcut *keyAccel*)

Methods

- Disable or enable a menu item by using:
 - void `setEnabled(boolean enabledFlag)`
 - boolean `isEnabled( )`
- Label set and get using:
 - void `setLabel(String newName)`
 - String `getLabel( )`
- Checkable menu item by using a subclass of **MenuItem** called **CheckboxMenuItem**. :
 - `CheckboxMenuItem( )`
 - `CheckboxMenuItem(String itemName)`
 - `CheckboxMenuItem(String itemName, boolean on)`

Methods

- Status about checkable MenuItem:
 - boolean getState()
 - void setState(boolean *checked*)
- For add MenuItem:
 - MenuItem add(MenuItem *item*)
- For add MenuBar
 - Menu add(Menu *menu*)
- To get Item from Menu:
 - Object getItem()

```

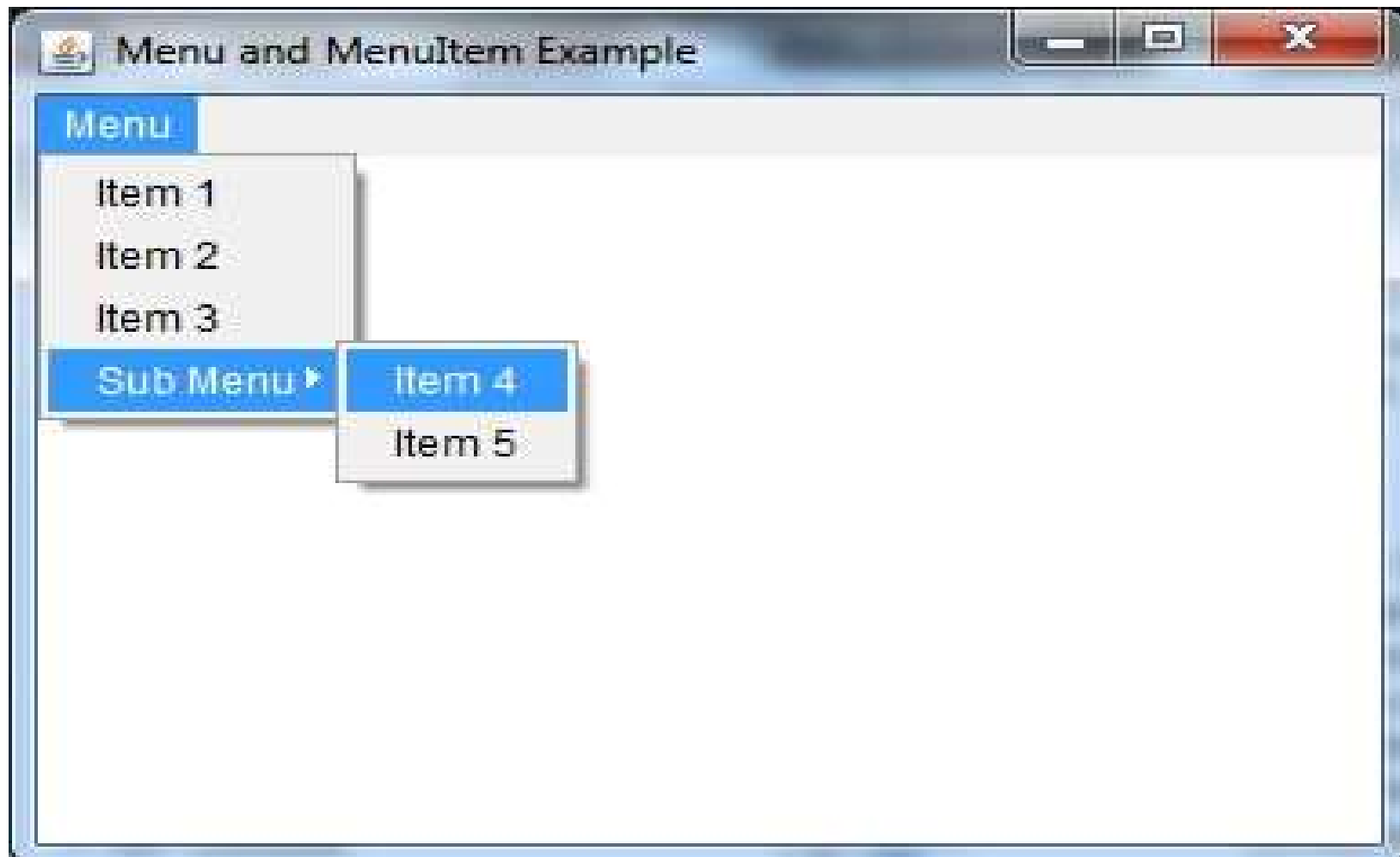
import java.awt.*;

class MenuExample
{
    MenuExample(){
        Frame f= new Frame("Menu Example");
        MenuBar mb=new MenuBar();
        Menu menu=new Menu("Menu");
        Menu submenu=new Menu("Sub Menu");
        MenuItem i1=new MenuItem("Item 1");
        MenuItem i2=new MenuItem("Item 2");
        MenuItem i3=new MenuItem("Item 3");
        MenuItem i4=new MenuItem("Item 4");
        MenuItem i5=new MenuItem("Item 5");

        menu.add(i1);
        menu.add(i2); menu.add(i3); submenu.add(i4);
        submenu.add(i5);
        menu.add(submenu);
        mb.add(menu);
        f.setMenuBar(mb);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }

    public static void main(String args[]) {
        new MenuExample();
    } }

```



DialogBox

- Dialog boxes are primarily used to obtain user input.
- They are similar to frame windows, except that dialog boxes are always child windows of a top-level window.
- Dialog boxes don't have menu bars.
- In other respects, dialog boxes function like frame windows.
- Dialog boxes may be modal or modeless.
- When a modal dialog box is active, all input is directed to it until it is closed.
- When a modeless dialog box is active, input focus can be directed to another window in your program.

Constructors:.

Dialog(Frame parentWindow, boolean mode)

Dialog(Frame parentWindow, String title, boolean mode)

To create Dialog Box:

Create Frame or Applet

Create another class which extends Dialog class.

Call this new class from Frame/Applet class.

In constructor of Extended Dialog class, use super method and pass vales to constructor of Dialog


```
import java.awt.*;
import java.awt.event.*;
public class DialogExample {
    private static Dialog d;
    DialogExample() {
        Frame f= new Frame();
        d = new Dialog(f , "Dialog Example", true);
        d.setLayout( new FlowLayout() );
        Button b = new Button ("OK");
        b.addActionListener ( new ActionListener()
        { public void actionPerformed((ActionEvent e )
            { DialogExample.d.setVisible(false); }
        });
        d.add( new Label ("Click button to continue."));
        d.add(b);      d.setSize(300,300);
        d.setVisible(true);    }
```



FileDialog

Java provides a built-in dialog box that lets the user specify a file.

To create a file dialog box, instantiate an object of type `FileDialog`.

Constructor:

`FileDialog(Frame parent, String boxName)`

`FileDialog(Frame parent, String boxName, int how)`

`FileDialog(Frame parent)`

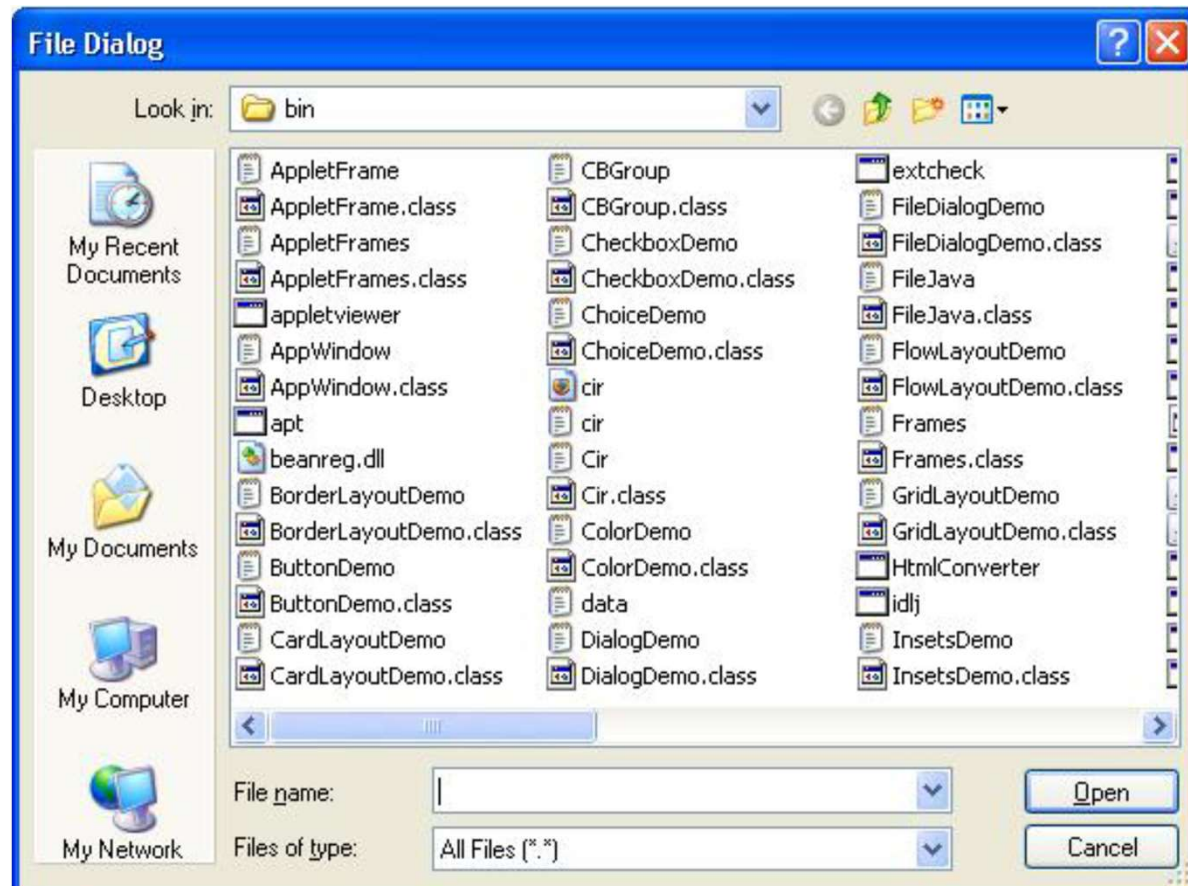
Int how: `FileDialog.LOAD`, `FileDialog.SAVE`

Methods:

`String getDirectory( )`

`String getFile( )`

```
import java.awt.*;
class SampleFrame extends Frame
{
    SampleFrame(String title){
        super(title); }
    class FileDialogDemo
    {public static void main(String args[]){
        Frame f = new SampleFrame("File Dialog Demo");
        f.setVisible(true);
        f.setSize(100, 100);
        FileDialog fd = new FileDialog(f, "File Dialog");
        fd.setVisible(true);
    }
}
```



Introduction to Java Swing



<https://www.javatpoint.com/java-swing>

What is Java Swing?

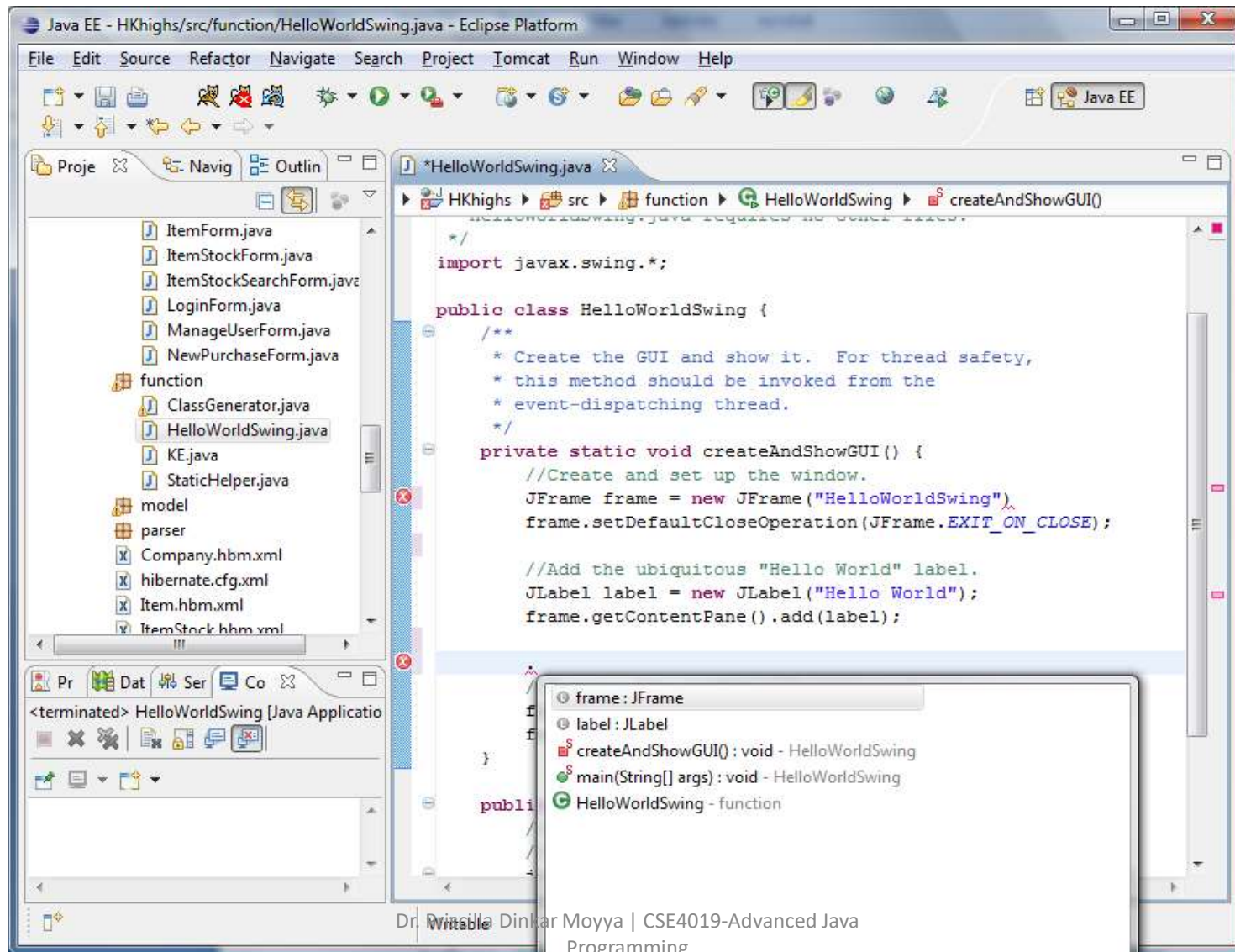
- Part of the Java Foundation Classes (JFC)
- Provides a rich set of GUI components
- Used to create a Java program with a graphical user interface (GUI)
- table controls, list controls, tree controls, buttons, and labels, and so on...

What features are available?

- GUI components like button, checkbox, and so on...
- Java 2D API: images, figures, animation
- Pluggable look and feel: use samples or create your own
- Data Transfer: cut, copy, paste, drag & drop
- Internationalization: supports different input language, right to left reading
- Accessibility API: for people with disabilities
- Undo Framework API: supports unlimited numbers of actions to undo and redo
- Flexible Deployment: run within a browser as an applet or Java Web Start

1. Visibility of system status

- This may be a strong advantage of Java Swing over other UI toolkits, not because of Java Swing itself is great, but because Eclipse provides such sophisticated checking on what is going on now
- Constantly checks for syntax errors
- Lists available methods or variables when press '.' (dot)
- However, you don't have synchronous result view
- You will have to run it in order to see the status of your program

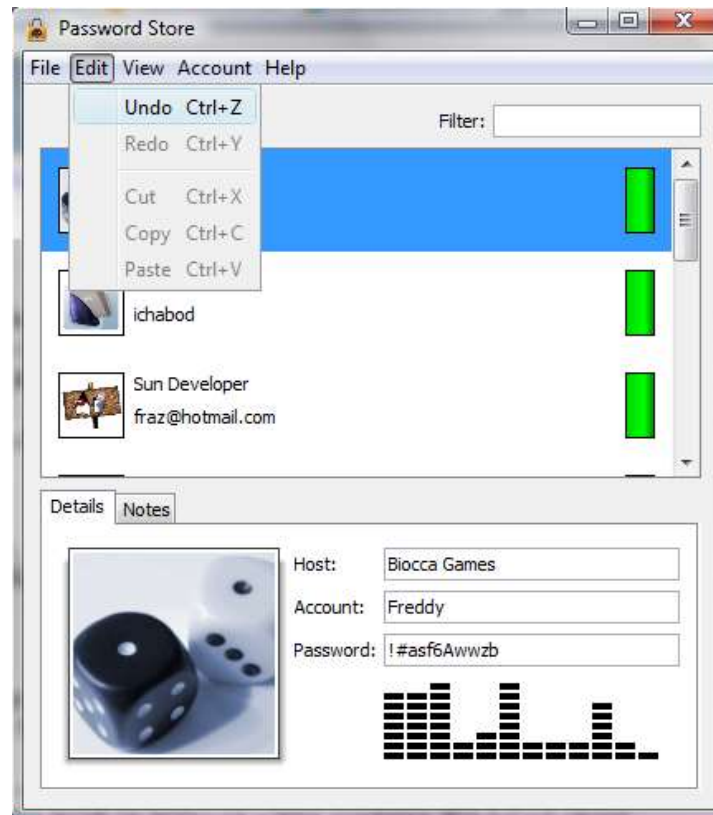


2. Match between system and the real world

- First of all, it is Java
- It follows Java convention
- It consists of 18 public packages of Java classes
- Its classes and methods are reasonably named
- Unless you are the first time programmer, you don't have to worry about its syntax or convention
- `JLabel developerNameLabel = new javax.swing.JLabel();`
`developerNameLabel.setFont(new java.awt.Font("Arial", 0, 14));`
`developerNameLabel.setForeground(new java.awt.Color(255, 255, 255));`

3. User control and freedom

- Eclipse supports strong undo and redo features
- You can't possibly go wrong and have to rewrite every code
- You can always fix it even though it may take you some effort and time
- Java Swing also provides undo and redo package
- `javax.swing.event.UndoableEditEvent;`
`javax.swing.event.UndoableEditListener;`
`javax.swing.undo.UndoableEdit;`

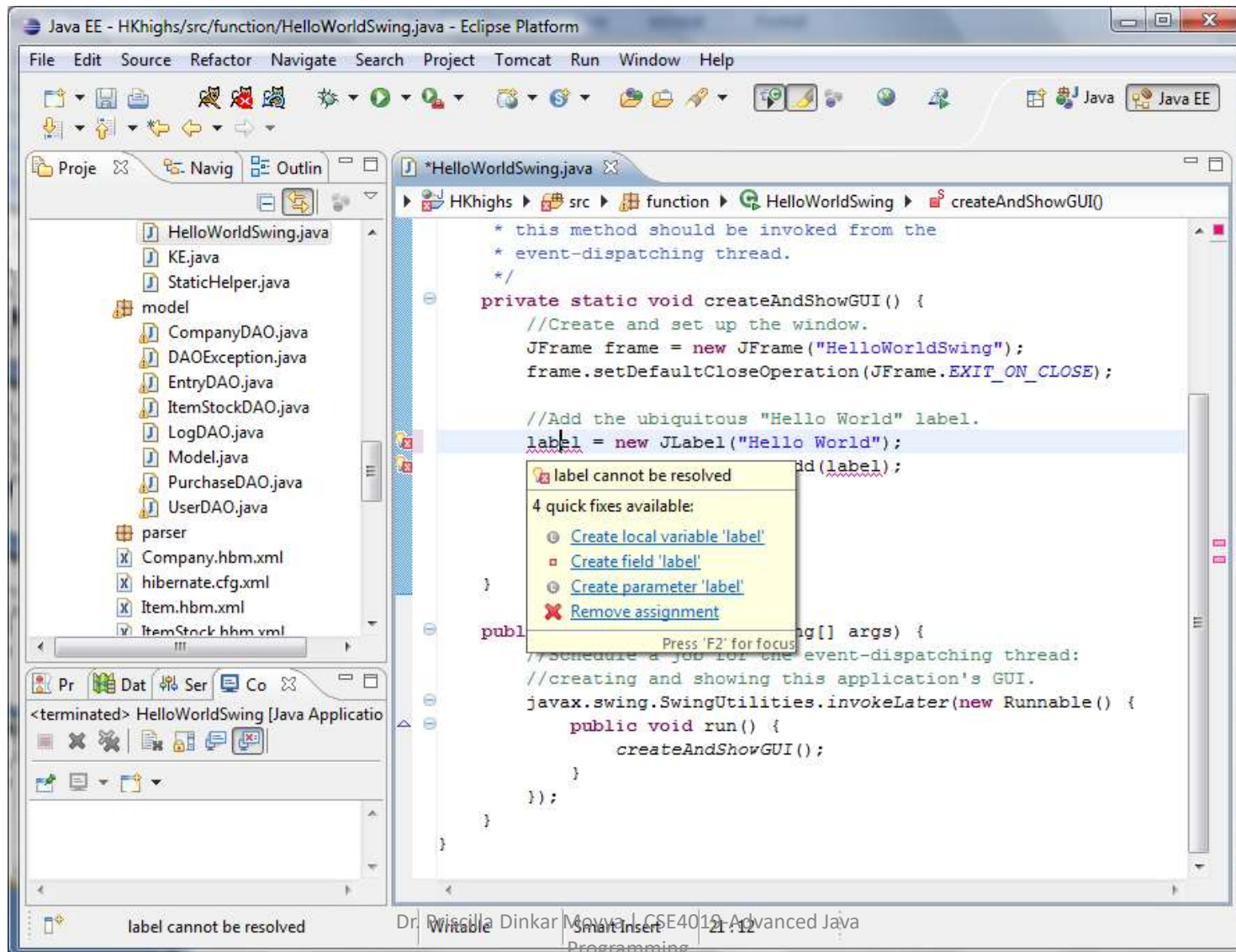


4. Consistency and standards

- Similar to #2
- Java Swing follows Java convention
- Packages, classes, methods, parameters, variables
- Mainly constructor, getter, setter

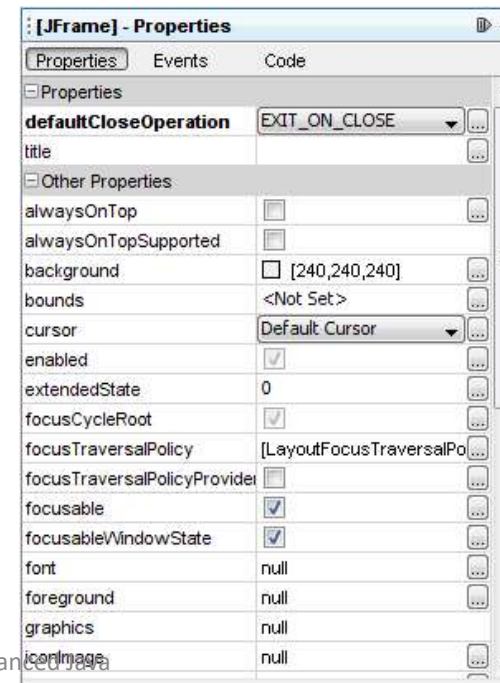
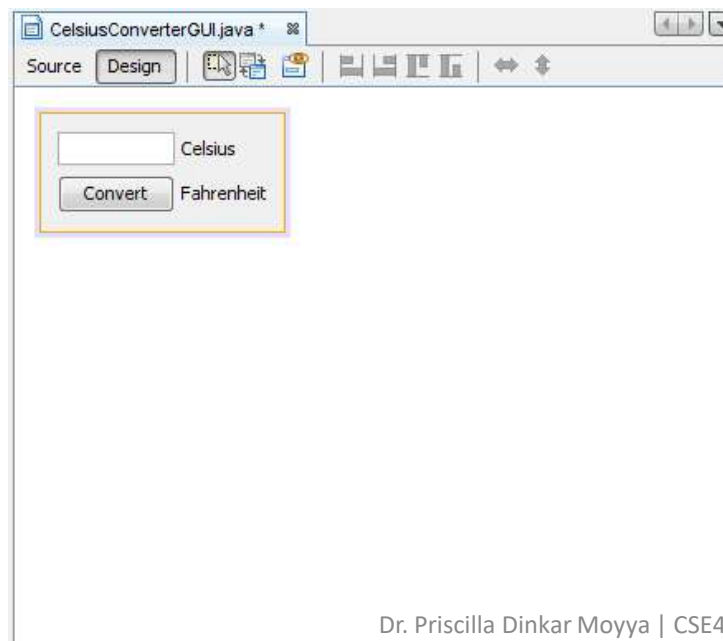
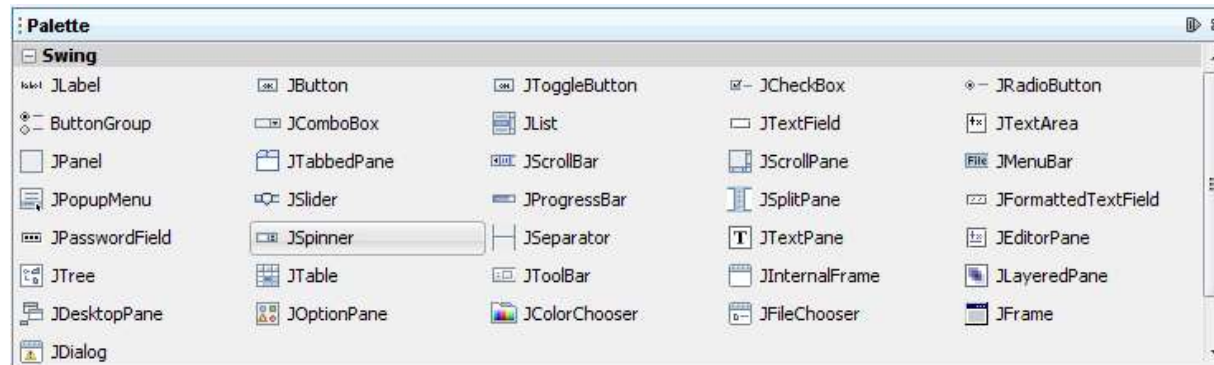
5. Error prevention

- First of all, Java is a strongly typed language: primitives and objects
- Eclipse checks for syntax and type errors continuously
- It gives red underline to errors and small red box to let you know which line in document it is located
- If you move your mouse over the error, then it suggests possible solutions to the error
- If you think one of the suggestions is a solution, then simply click on it to apply it
- Of course, it won't compile and run until you fix all the syntax errors
- However, you don't have any idea about runtime errors except that you will have to run it and find out



6. Recognition rather than recall

- It's a programming language, so it's better and faster for you if you memorize names and their functions of classes or methods
- However, whenever you press 'dot' after name of package, class, or object, then eclipse provides you a list of all possible subclasses, functions and variables
- If you move your mouse over almost anything, then eclipse provides you with a text document associated with it, usually javadoc, or you can click on it and it directs you to online javadoc page
- You don't have a help of graphical interface to develop a graphical interface, so it maybe a disadvantage of Java Swing with eclipse.
- By the way, you can have a graphical interface if you use NetBeans IDE instead of eclipse



7. Flexibility and efficiency of use

- Swing includes many basic components as a package, so it is efficient to use them
- At the same time, you can create almost anything you want as combination of those components and some pure coding in Java
- Java have had a reputation for being slower and requiring more memory than those written in natively compiled languages such as C or C++
- However, the performance heavily depends on how you optimize your codes and which components of UI you use the most frequently
- It may be subsequently slower or faster

8. Aesthetic and minimalist design

- Swing is designed in a way that it provides a set of "lightweight" (all-Java language) components that, to the maximum degree possible, work the same on all platforms
- It includes almost only and all components we could find around any software with user interface
- Yet, it gives developers varieties to customize those components

9. Help users recognize, diagnose, and recover from errors

- Syntax and type checking errors are already covered previously
- Java shows where in the code runtime errors (exceptions) are caused, specifying line numbers and brief reason for error on the console of eclipse
- It's not always right, but most of the times it is right
- It's relatively easy to find the cause of the error and debug it comparing to other languages I have experienced with

10. Help and documentation

- Javadoc
- Eclipse's support for javadoc (already covered)

AWT vs. Swing

Abstract Windowing Toolkit (AWT)

- Original Java GUI toolkit
- Wrapper API for native GUI components
- Lowest-common denominator for all Java host environments

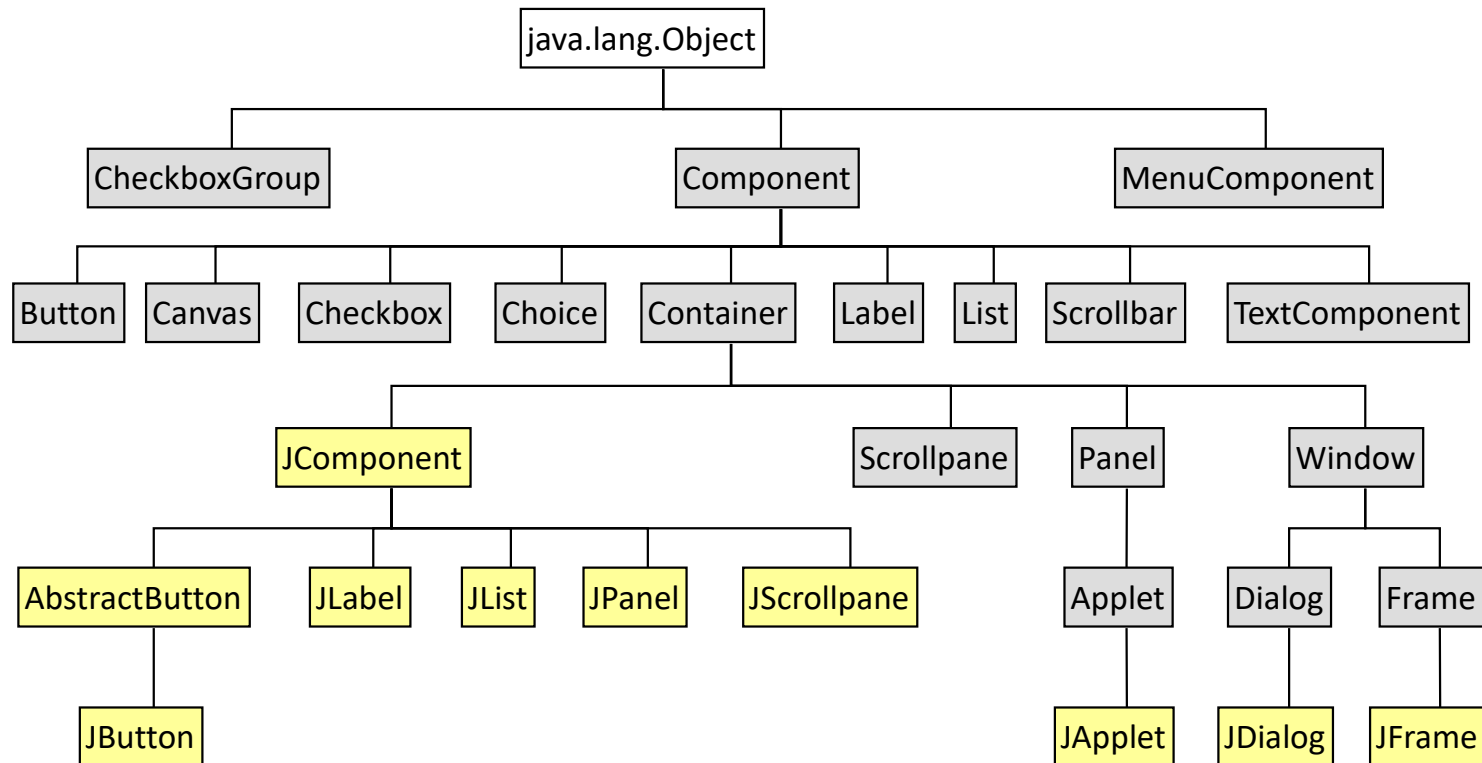
Swing

- Implemented entirely in Java on top of AWT
- Richer set of GUI components
- Pluggable look-and-feel support

AWT vs. Swing

No.	Java AWT	Java Swing
1)	AWT components are platform-dependent .	Java swing components are platform-independent .
2)	AWT components are heavyweight .	Swing components are lightweight .
3)	AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
4)	AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT doesn't follows MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC .

Partial AWT and Swing Class Hierarchy



Swing Hello World

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class HelloWorld {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Hello World!");
        frame.setSize(220, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        Container contentPane = frame.getContentPane();
        contentPane.setLayout(null);

        JButton button = new JButton("Hello World!");
        button.setLocation(30, 30);
        button.setSize(150, 100);
        contentPane.add(button);

        frame.setVisible(true);
    }
}
```



Swing Hello World with Events

```
...
public class HelloWorld {
    public static void main(String[] args) {
        ...
        JButton button = new JButton("Hello World!");
        button.addActionListener(new MyActionListener());
        ...
    }
}
```

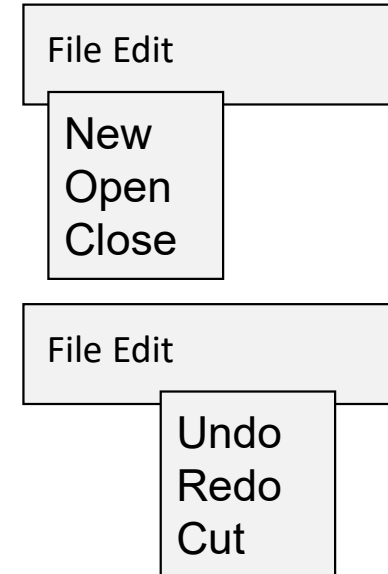
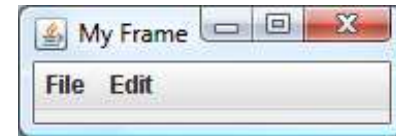
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class MyActionListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        Toolkit.getDefaultToolkit().beep();
    }
}
```

Containment Hierarchy of a Menu

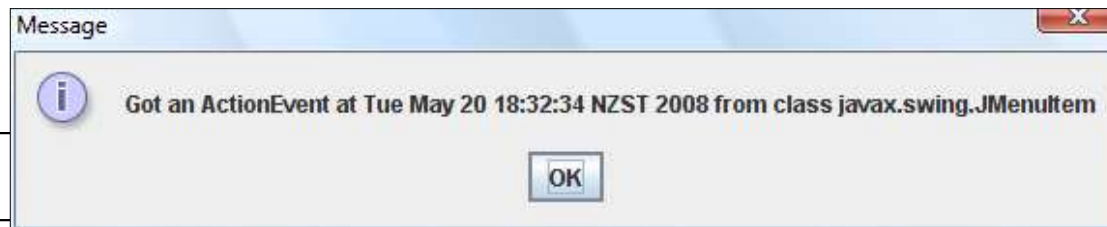
...

```
public class MenuExample {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("My Frame");  
        frame.setDefaultCloseOperation(  
            JFrame.EXIT_ON_CLOSE);  
  
        JMenu fileMenu = new JMenu("File");  
        fileMenu.add(new JMenuItem("New"));  
        fileMenu.add(new JMenuItem("Open"));  
        fileMenu.add(new JMenuItem("Close"));  
  
        JMenu editMenu = new JMenu("Edit");  
        editMenu.add(new JMenuItem("Undo"));  
        editMenu.add(new JMenuItem("Redo"));  
        editMenu.add(new JMenuItem("Cut"));  
  
        JMenuBar menubar = new JMenuBar();  
        menubar.add(fileMenu);  
        menubar.add(editMenu);  
  
        frame.setJMenuBar(menubar);  
        frame.setVisible(true);  
    }  
}
```



Handling Menu Events

```
...
public class MenuExample {
    static JFrame frame;
    public static void main(String[] args) {
        ...
        JMenuItem item = new JMenuItem("Close");
        item.addActionListener(new MenuActionListener());
        fileMenu.add(item);
        ...
    } }
```



```
...
public class MenuActionListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(MenuExample.frame,
            "Got an ActionEvent at " + new Date(e.getWhen())
            + " from " + e.getSource().getClass());
    } }
```

Different Kinds of Swing Events

Low-level events

- **MouseEvent**: Component got mouse-down, mouse-move, etc.
- **KeyEvent**: Component got key-press, key-release, etc.
- **ComponentEvent**: Component resized, moved, etc.
- **ContainerEvent**: Container's contents changed because a component was added or removed
- **FocusEvent**: Component got focus or lost focus
- **WindowEvent**: Window opened, closed, etc.

High-level semantic events

- **ActionEvent**: Main action of control invoked (e.g. JButton click)
- **AdjustmentEvent**: Value was adjusted (e.g. JScrollBar moved)
- **ItemEvent**: Item was selected or deselected (e.g. in JList)
- **TextEvent**: Text in component has changed (e.g. in JTextField)

Events, Listeners, Adapters and Handler Methods

Event	Listener / Adapter	Handler Methods
ActionEvent	ActionListener	actionPerformed
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged
MouseEvent	MouseListener MouseAdapter	mouseClicked mouseEntered mouseExited mousePressed mouseReleased
KeyEvent	KeyListener KeyAdapter	keyPressed keyReleased keyTyped
ComponentEvent	ComponentListener ComponentAdapter	componentShown componentHidden componentMoved componentResized

JLabel

- Label
 - Provide text on GUI
 - Defined with class JLabel
 - Can display:
 - Single line of read-only text
 - Image
 - Text and image
 - `setToolTipText` method (from `JComponent`) specifies text that appears when user moves cursor over JLabel

TextFields

- **TextField**
 - Single-line area in which user can enter text
 - `setEditable(false)`, user cannot modify textfield
- **JPasswordField**
 - Extends `TextField`
 - Hides characters that user enters

JTextArea

- JTextArea
 - Area for manipulating multiple lines of text
 - Constructors
 - JTextArea(string, rows, cols)
 - JTextArea(rows, cols)
 - Since JTextAreas do not provide scrollbars, usually are placed in JScrollPane objects

JButton

- Button
 - Component user clicks to trigger a specific action
 - Several different types
 - Command buttons
 - Check boxes
 - Toggle buttons
 - Radio buttons
 - `javax.swing.AbstractButton` subclasses
 - Command buttons are created with class `JButton`
 - Text on button face called “button label”
 - Button can have icon and rollover icon
 - Generate `ActionEvents` when user clicks button
 - `getActionCommand` returns button label

JCheckBox and JRadioButton

- State buttons
 - On/Off or true/false values
 - Java provides three types
 - JToggleButton
 - JCheckBox
 - JRadioButton
 - getSource returns button object that generated event
 - isSelected tells if a JCheckBox has been selected or not
 - JRadioButtons normally appear as a ButtonGroup
 - JRadioButtons are mutually exclusive (old time radio buttons)

JComboBox

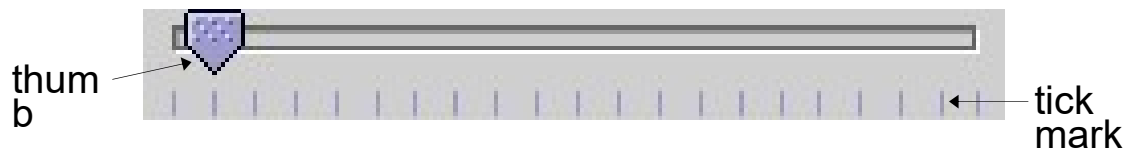
- JComboBox
 - List of items from which user can select
 - Also called a *drop-down list*
 - `setMaximumRowCount` sets max number of elements displayed when user clicks on combo box
 - `getSelectedIndex` tells which item in the combo box was selected

JList

- List
 - Series of items
 - user can select one or more items
 - single-selection and multiple-selection lists
 - JList does not provide scrollbar, JScrollPane necessary
 - `getSelectedIndex` tells which item in the list was selected

JSlider

- JSlider
 - Enable users to select from range of integer values
 - Several features
 - Tick marks (major and minor)
 - Snap-to ticks
 - Orientation (horizontal and vertical)
 - paint draws on subclasses of JFrame and JApplet
 - paintComponent draws on subclasses of JComponent



Using Menus with Frames

- JMenu
 - Allows for performing actions without cluttering GUI
 - Can be used for JFrame or JApplet objects
 - Contained by menu bar
 - JMenuBar (requires setJMenuBar)
 - Comprised of menu items
 - JMenuItem
 - JCheckBoxMenuItem
 - JRadioButtonMenuItem
 - When menu is clicked, expands to show menu items
 - JMenuItem can be a JMenu to have a sub-menu
 - addActionListener for each menu item

Jprogressbar

- Progress Bar is an horizontal or vertical bar that visualize the progress of an operation.
- The class that is used to render the progress bar is 'JProgressBar' that is the part of the Java Swing package.
- It displays the progress of specific task by filling of the bar and/or displaying the percentage of completion.
- It may even display the text as specified. As the task reaches its completion, the progress bar fills up.

Mouse Event Handling

- Event-listener interfaces for mouse events
 - `MouseListener`
 - `MouseMotionListener`
 - `MouseEvent` object tells what event occurred and the x and y coordinates (`e.getX()` and `e.getY()`)

MouseListener and MouseMotionListener interface methods

MouseListener and MouseMotionListener interface methods	
<i>Methods of interface MouseListener</i>	
<code>public void mousePressed(MouseEvent event)</code>	
	Called when a mouse button is pressed while the mouse cursor is on a component.
<code>public void mouseClicked(MouseEvent event)</code>	
	Called when a mouse button is pressed and released while the mouse cursor remains stationary on a component.
<code>public void mouseReleased(MouseEvent event)</code>	
	Called when a mouse button is released after being pressed. This event is always preceded by a mousePressed event.
<code>public void mouseEntered(MouseEvent event)</code>	
	Called when the mouse cursor enters the bounds of a component.
<code>public void mouseExited(MouseEvent event)</code>	
	Called when the mouse cursor leaves the bounds of a component.
<i>Methods of interface MouseMotionListener</i>	
<code>public void mouseDragged(MouseEvent event)</code>	
	Called when the mouse button is pressed while the mouse cursor is on a component and the mouse is moved while the mouse button remains pressed. This event is always preceded by a call to mousePressed. All drag events are sent to the component on which the drag began.
<code>public void mouseMoved(MouseEvent event)</code>	
	Called when the mouse is moved when the mouse cursor on a component. All move events are sent to the component over which the mouse is currently positioned.

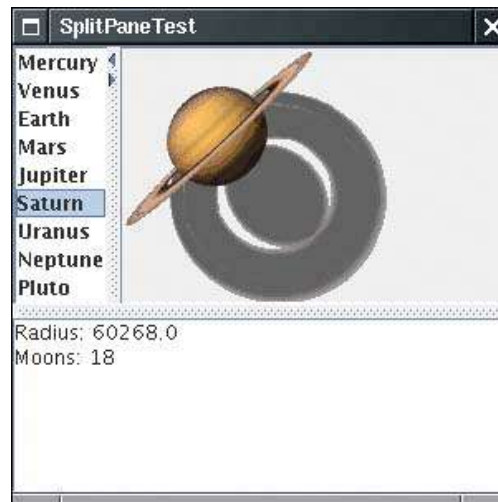
Key Event Handling

- Interface KeyListener (KeyAdapter)
 - Handles *key events*
 - Generated when keys on keyboard are pressed and released
 - KeyEvent
 - Contains *virtual key code* that represents key

Component Organizers

- Split Panes

- Split panes split a component into two parts, with an adjustable boundary in between.
- The outer pane is split vertically, with a text area on the bottom and another split pane on the top.
- That pane is split horizontally, with a list on the left and a label containing an image on the right.



Dr. Priscilla Dinkar Moyya | CSE4019-Advanced Java Programming

Component Organizers

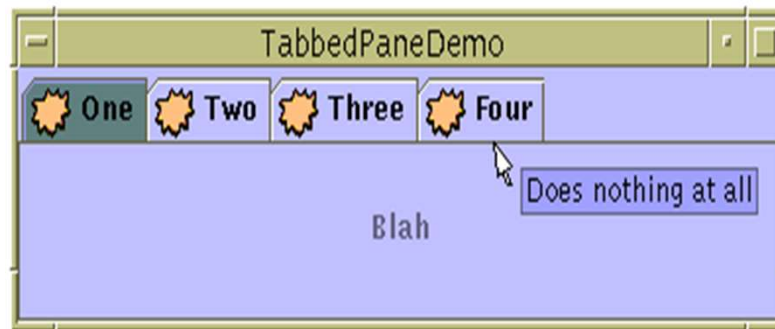
- Split Panes

- You construct a split pane by specifying the orientation, one of `JSplitPane.HORIZONTAL_SPLIT` or `JSplitPane.VERTICAL_SPLIT`, followed by the two components. For example,
 - `JSplitPane innerPane = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, planetList, planetImage);`
 - `innerPane.setOneTouchExpandable(true);`
 - `innerPane.setContinuousLayout(true);`

Component Organizers

- **Tabbed Panes**

- With the JTabbedPane class, you can have several components share the same space.
- The user chooses which component to view by selecting the tab corresponding to the desired component. If you want similar functionality without the tab interface, you might want to use a card layout instead of a tabbed pane.
- To create a tabbed pane, you simply instantiate JTabbedPane, create the components you wish it to display, and then add the components to the tabbed pane using the addTab method.
- Here is a picture of an application that has a tabbed pane with four tabs:



Component Organizers

- Desktop Panes

- The JDesktopPane class, can be used to create "multi-document" applications.
- A multi-document application can have many windows included in it.
- We do it by making the contentPane in the main window as an instance of the JDesktopPane class or a subclass.
- Internal windows add instances of JInternalFrame to the JdesktopPane instance.
- The internal windows are the instances of JInternalFrame or its subclasses.

JDesktopPane()	Creates a new JDesktopPane.
----------------	-----------------------------

Modifier and Type	Field	Description
static int	LIVE_DRAG_MODE	It indicates that the entire contents of the item being dragged should appear inside the desktop pane.
static int	OUTLINE_DRAG_MODE	It indicates that an outline only of the item being dragged should appear inside the desktop pane.

Component Organizers

- Internal Panes

- JFrame is a container that provides many features of a frame which includes displaying title, opening, closing, resizing, support for menu bar, etc.
- JFrame() : creates a new non- closable, non- resizable, non- iconifiable, non- maximizable JFrame with no title
- JFrame(String t) :creates a new non- closable, non- resizable, non- iconifiable, non- maximizable JFrame with a title specified
- JFrame(String t, boolean resizable) :creates a new non- closable, non- iconifiable, non- maximizable JFrame with a title and resizability specified
- JFrame(String t, boolean resizable, boolean closable) : creates a new non- iconifiable, non- maximizable JFrame with a title, closability and resizability specified
- JFrame(String t, boolean resizable, boolean closable, boolean maximizable) :creates a new non- iconifiable JFrame with a title, closability, maximizability and resizability specified
- JFrame(String t, boolean resizable, boolean closable, boolean maximizable, boolean iconifiable) : creates a new JFrame with a title, closability, maximizability, iconifiability and resizability specified

Component Organizers

- Internal Panes : Commonly used methods
 - setFrameIcon(Icon icon) : sets the icon for the frame to the specified image
 - setLayout(LayoutManager manager) : sets the layout of the frame to specified layout manager
 - setTitle(String t): set the title of the frame to specified title
 - getTitle() : get the title of the frame
 - reshape(int x, int y, int width, int height) : resize the frame to specified width and height and a specified location
 - add(Component c) : adds the specified component to the container.
 - addImpl(Component c, Object co, int i) : adds the specified component.
 - addInternalFrameListener(InternalFrameListener l) : adds the specified InternalFrameListener to the list.
 - createRootPane() : called by the constructor to set up the JRootPane.

END